QuantSim — Architecture & Physical Design

Document	Architecture & Physical Design (ARC)
System	QuantSim — AI-Assisted Market Simulation & Portfolio Risk Analytics Platform
Status	**Proposed — for team review.** The architectural style is settled (microservices, §3); everything downstream of it remains open to challenge.
Version	0.2 (draft) — supersedes 0.1; architectural style changed from modular monolith to **microservices** by team decision. Revised after an audit against the approved seven-module proposal: source authority fixed (§0), worker-deployment count reconciled to seven (§5.2), rolling-correlation ownership settled (AD-19), cleaning made a publication gate (AD-20), authorization placement recorded as a decision (AD-18)
Date	2026-08-23
Companion	*QuantSim — Requirements (FR/NFR)* — separate document

0. How to read this document, and the assumptions it rests on

This is the first of two specifications. It defines **how QuantSim is built and deployed** — the logical architecture, the runtime/execution model, the communication fabric, the physical data model, and the deployment topology. The companion document defines **what it must do** (functional requirements) and **how well** (non-functional requirements). Where this document makes a design choice, it names the choice, gives the mechanism, and states *why it is optimal for this system* — and, where a reasonable person would choose differently, it records the rejected alternative and the reason for rejecting it. Sections are numbered so the requirements document and future design notes can cite them precisely (e.g. "ARC §7.3", "ARC §8.6").

What constrains this design, and what does not. The **approved seven-module project proposal** is the binding statement of scope. Its module list and its numbering are carried through this document unchanged (§5), and the companion document's §4.1 shows every capability it names reaching a requirement. This document adds to the proposal — identity, audit, an outbox, read models — but never contradicts it.

The team's **internal brainstorm** on structure is a *non-binding* input. It was a useful source of candidate detail and much of it survives on merit; it is not a design authority, and where it proposed a structure this document adopts it only where it is the efficient answer. Four of its structural positions are rejected, each for a stated engineering reason rather than a stylistic one, and each rejection is argued where it arises: the **single six-layer stack** in which every layer talks only to the layer below (§4 — under microservices layering lives inside a service, repeated eight times, and the system level is a topology, not a stack); the **event backbone placed as a layer between the domain and the data-access layer** (§4.1 Tier 4, §7.4 — Kafka is a decoupling and durability mechanism deliberately off the synchronous read path, not a hop every request passes through; a request that had to traverse a log to reach its own database would be slower and less reliable than one that did not); **all authorization decided at the API layer** (§9.2 — authentication belongs at the edge, but a gateway-only authorization check means any request arriving by another route, including an event

consumer or an internal call, is implicitly trusted); and **eight schemas inside one database** (§8.2 — a shared instance leaves every service one convenient `JOIN` from a distributed monolith and shares a failure domain besides). Where the brainstorm and the proposal disagree, the proposal wins; where the brainstorm goes beyond the proposal, the addition has to justify itself here.

Three baseline positions govern everything that follows. **A1 is a settled team decision; A2 and A3 remain decisions to confirm rather than facts.**

A1 — Architectural style: microservices. The seven proposal modules are built and deployed as **seven independently deployable services**, fronted by an API gateway and supported by a Platform/Identity service — nine deployable units in total (§3, §5). Each service owns its own database, and services communicate through a Kafka event backbone for asynchronous facts and versioned REST/gRPC contracts for synchronous reads (§7). This is the team's deliberate choice and it supersedes the modular-monolith baseline that version 0.1 of this document recommended.

The properties being bought are real. Each service deploys on its own cadence without redeploying the platform, so a change to the TDA pipeline cannot break trade execution. The three compute-heavy services (Simulation, ML, Risk & TDA) scale — and fail — independently of the interactive path. Each service may choose the runtime, libraries, and storage layout its workload actually needs rather than a lowest-common-denominator shared stack. A fault in one service degrades one capability instead of the whole application. And the team gains direct experience of the architecture that dominates industry practice, which is a legitimate objective for a final-year project.

The costs are equally real and are accepted knowingly rather than argued away. Data becomes distributed: there are no cross-service joins and no distributed transactions, so any view spanning two services must be assembled by the caller or maintained as a local read model (§8.2, §8.12). Consistency becomes eventual, mediated by choreographed sagas and a transactional outbox instead of a single `COMMIT` (§7.6, §7.7). The network becomes an ordinary failure condition, requiring timeouts, retries with jitter, circuit breakers, and bulkheads on every hop (§7.5). And the operational surface grows to nine deployable units with their own contracts, pipelines, dashboards, and traces (§9.5, §10). The remainder of this document specifies the mechanisms that make these costs manageable; it does not pretend they are absent.

A2 — Deployment target: Kubernetes in production, Docker Compose for development and the demo. Microservices need an orchestrator to be operated honestly — nine deployable units with independent lifecycles, health checks, and scaling policies are what Kubernetes exists for — so Kubernetes is the specified production target (§10.2). The day-to-day development and demonstration runtime remains a single-machine Docker Compose stack that brings up all nine units — sixteen application containers once each service's worker is counted (§10.1) — plus their infrastructure. Two concessions make that tractable on a laptop and are stated here rather than buried: the per-service databases run as **separate logical databases on one shared PostgreSQL instance** in development, which preserves the ownership rule (no service reads another's tables, no cross-service joins) while consuming one process instead of eight; and the Kafka backbone runs as a single broker locally rather than a three-broker cluster. Neither concession changes any service's code or contract — only the connection string and the broker's replication settings differ between environments, which is what keeps the local-to-production path short.

A3 — Physical data model lives here. Because this document is titled *physical* design, the concrete data-at-rest model belongs in it: the per-service database boundaries, the load-bearing tables, the time-series storage strategy, partitioning, indexing, retention, numeric precision, and the two subsystems whose data

model *is* the hard part — the append-only trading ledger and the point-in-time-correct feature store. It also covers what distributed data forces into the design: the transactional outbox that makes "write my state and publish my event" atomic without a distributed transaction, and the local read models each service maintains from other services' events. The requirements document will reference these by name rather than redefining them (§8).

Two further baseline choices worth stating early, both revisited with justification later: every service is **Python 3.12 + FastAPI** on an **async** stack unless a workload justifies otherwise (§13), and the event backbone is **Apache Kafka in every environment** — a three-broker cluster in production, a single broker locally (§7.4). Version 0.1 of this document allowed Redis Streams as a local substitute; that is withdrawn, because under microservices the backbone carries cross-service consistency (the outbox drain, saga choreography, and consumer offsets in §7.6–§7.7) and dev must therefore exercise the same broker semantics as production.

1. What the architecture has to serve

Good architecture is derived from forces, not from a component wish-list. Seven forces dominate QuantSim and every major decision in this document traces back to one of them.

Two heavy, bursty compute paths. A Monte Carlo run generates *N* paths over *T* steps and can occupy a core for minutes; ML model training can occupy a GPU for far longer. These are CPU/GPU-bound, they arrive in bursts (a user kicks off several at once), and they must **not** be allowed to block interactive traffic. This force alone justifies out-of-process workers and a job queue (§6).

A heavy time-series write load. OHLCV bars, simulated price paths (potentially millions of rows per run), tick and order events during a replay, and portfolio value series are all append-heavy time series. A general-purpose relational layout would bloat and slow down. This force justifies TimescaleDB hypertables, compression, and continuous aggregates (§8.4).

Auditability and exact replay are first-class, not features. The domain demands that a portfolio's entire history be reconstructable to the event, that every business write be recorded with before/after state, and that audit survive even the deletion of the underlying row. This force justifies event sourcing for the trading domain and a trigger-based audit subsystem (§8.5, §8.7).

The dashboard is read-heavy and latency-sensitive. Widgets must return in about two seconds, and reads vastly outnumber writes. This force justifies a read/write split (a read replica serving the dashboard), aggressive precomputation, and a cache tier (§8.8, §9.4).

Correctness has sharp, domain-specific edges. Financial ML fails silently under look-ahead bias; money fails silently under floating-point drift; a simulation is meaningless if run against a calibration fitted to a different instrument. These are not "nice to have" — they are correctness invariants the *data layer* must enforce, not merely the application (§8.6, §8.9).

Capabilities must evolve and fail independently. The seven domains progress at very different rates and carry very different risk: the TDA pipeline is research-grade and will churn, while trade execution must stay stable and correct; ML training needs GPUs and heavyweight libraries that nothing else needs. A change to one must not force a redeploy of the others, and a failure in one must degrade one capability rather than the platform. This force is what the microservices decision (A1, §3) answers directly, and it is the force that justifies paying the distributed-systems cost the other forces would not on their own.

A small team on a fixed timeline. The optimum is bounded by who is building it. Three people cannot operate nine deployable units the way a nine-team organization would, so the design deliberately trades some microservices orthodoxy for operability: one shared service template so every service is structured identically (§5.1), infrastructure co-located in development (A2), choreographed sagas rather than a bespoke orchestrator (§7.6), and no service mesh until something demands one (§10.2, §15). Every mechanism in this document is chosen to be the best option *this* team can actually run.

2. Architectural principles

The following principles are applied consistently throughout; later sections refer back to them.

P1 — Simulation is the core; everything else is instrumentation around it. The market environment (replayed-historical *or* synthetic) is the substrate that the trading engine, ML, risk, and TDA all consume. The architecture makes the *market environment* an explicit, first-class abstraction that the trading engine binds to without knowing which kind it is.

P2 — Separate the interactive plane from the compute plane. Anything a user waits on synchronously (CRUD, reads, launching a job, streaming results) is served by a service's API tier. Anything long-running (simulation, training, heavy risk/TDA) runs on that service's own worker deployment and reports back asynchronously. Under microservices this line is drawn *inside* each compute-heavy service — Simulation, ML, and Risk each ship an API deployment and a worker deployment from the same service (§6) — so a saturated worker pool never consumes the request-handling capacity of its own service, let alone anyone else's.

P3 — Append-only where history matters; derive the rest. Trade events, holdings history, portfolio value, audit, and predictions are recorded immutably. Current state (a portfolio's present positions, its latest value) is a *projection* of the append-only log and can always be rebuilt by replay.

P4 — Enforce invariants at the lowest layer that can hold them. Role sets, monetary precision, temporal validity (replay end > start), and the simulation-to-calibration instrument match are enforced in the database with types, constraints, and triggers — not left to application code that a second code path could bypass.

P5 — Configuration over migration for open-ended catalogs. New ML model types and new stress scenarios must be addable without a schema change, so those catalogs are modeled as registry rows with typed core columns plus a validated JSONB configuration payload.

P6 — Read the cheap thing, not the expensive thing. Dashboards read precomputed aggregates, materialized projections, and cache — never raw event logs or full path tables at request time.

P7 — Everything reproducible from code. Schema is migrations; infrastructure is Terraform; there is no state that exists only because an engineer once typed a command by hand.

P8 — A service owns its data absolutely. Every service holds the only credentials to its own database. No other service — and no analyst with a SQL client — reads or writes those tables directly; the only ways in are the service's published API and the events it emits. There are no shared tables, no cross-service foreign keys, and no cross-service joins. This is the single rule that makes the decomposition real rather than cosmetic: violate it once with a convenient direct query and the eight services collapse into a distributed monolith that must be deployed in lockstep. Where a service genuinely needs another's data on its read path, it maintains its own local projection fed by events (§8.12).

P9 — Design for partial failure, because it is the normal case. With nine deployable units, "some dependency is unavailable right now" is an expected operating condition, not an incident. Every synchronous call therefore carries a timeout, a bounded retry with jitter, and a circuit breaker; every consumer is idempotent because delivery is at-least-once; every cross-service workflow that spans two databases is a saga with an explicit compensating action; and every service defines what it does when a dependency is down — serve stale data, queue the work, or return a clearly-degraded response, but never hang and never silently lose the request (§7.5, §7.6, §11).

3. Architectural style decision

Decision: microservices — nine independently deployable units, each owning its own database, integrated by a Kafka event backbone for asynchronous facts and versioned synchronous contracts for reads. The seven proposal modules become seven **domain services**; a **Platform/Identity service** carries the cross-cutting concerns that belong to no domain (accounts, roles, audit); and an **API gateway** is the single edge the browser talks to. Each compute-heavy service (Simulation, ML, Risk/TDA) ships two deployments from one codebase — an API tier and a worker tier — so its long jobs never contend with its own request handling (P2, §6).

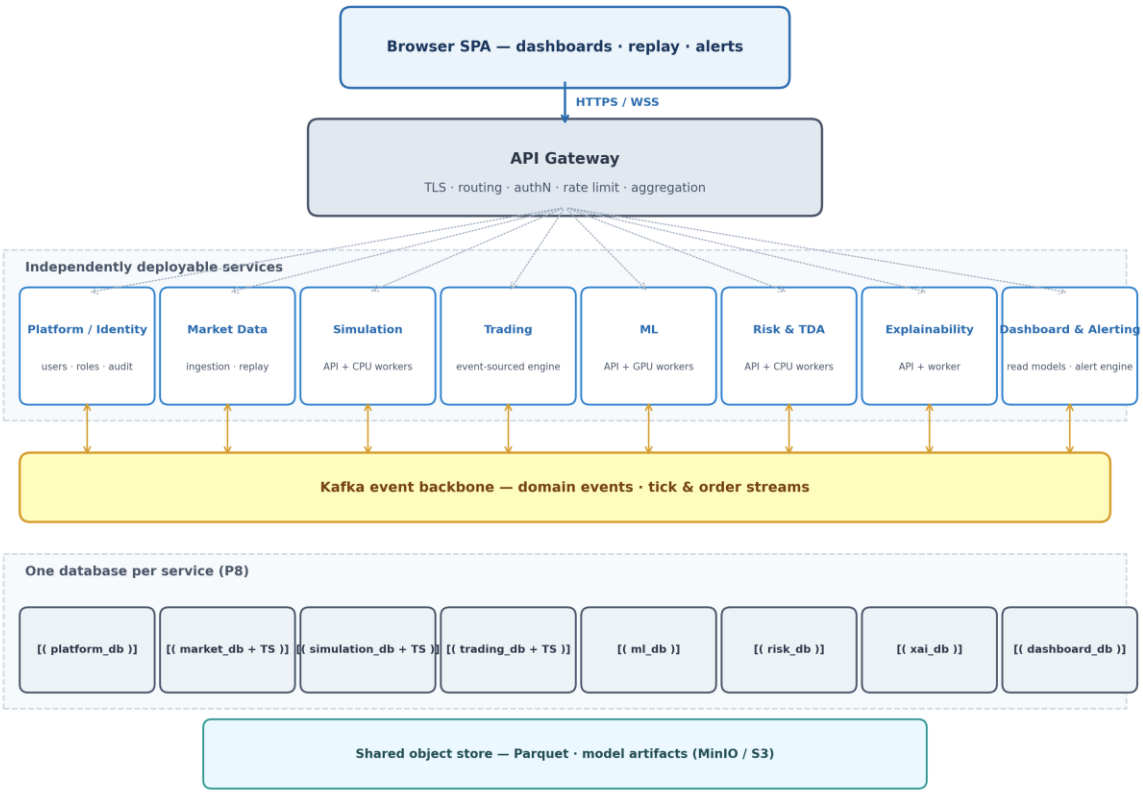


Figure 3.1: Microservices Architecture, API Gateway, Kafka Event Backbone, and Per-Service Databases

Where the service boundaries are drawn, and why there. The boundaries follow the **bounded contexts of the domain**, not the technical layers — there is no "database service" or "business-logic service", because a boundary drawn across a layer forces every feature to touch every service. Each of the seven domains owns a distinct language and a distinct lifecycle: Market Data speaks bars, baskets and replay sessions; Simulation

speaks calibrations and path ensembles; Trading speaks orders, fills and an event log; ML speaks models, features and predictions; Risk speaks scenarios, persistence diagrams and Betti numbers; XAI speaks attributions and narratives; Dashboard speaks widgets, layouts and alert rules. These vocabularies barely overlap, which is the empirical test that a boundary is in the right place. The result is a 1:1 correspondence between the proposal's modules, the services, and the databases — a property worth having for a project that must be explained and defended, because there is exactly one decomposition to learn.

What is deliberately *not* split. Calibration stays inside Simulation rather than becoming its own service, because a calibration is never useful without a run and the two share a transactional lifecycle. Stress testing and TDA stay inside one Risk service, because both consume the same correlation inputs and both feed the same composite risk view. The Platform service holds identity, RBAC policy, and the audit sink together because they are one governance concern. Splitting these further would produce services that cannot change independently — chatty boundaries with no autonomy, all of the cost of a network hop and none of the benefit.

Why this is the right shape given the decision. With the style settled, the remaining question is whether *this* decomposition delivers the properties A1 is buying, and it does: the TDA pipeline (the most volatile code in the system) deploys without touching trade execution; ML scales onto GPU nodes while nothing else needs a GPU in its image; Simulation absorbs a burst of Monte Carlo work by adding worker replicas while the dashboard stays responsive; and if the XAI service's external LLM provider is down, explanations degrade to cached text while every other capability is unaffected. Each of those is a concrete NFR in the companion document, and §12 traces them.

Rejected — modular monolith with out-of-process workers. This was version 0.1's recommendation and remains the lower-cost answer on paper: one deployable, one database, in-process calls, no eventual consistency, no saga compensation, and a single trace per request. It was rejected by team decision in favour of the independent-deployability, fault-isolation, per-service scaling, and learning objectives set out in A1. The rejection is recorded rather than erased because it is the honest counterfactual: a reviewer who asks "did you consider the simpler option?" should find the answer here, along with the specific benefits that justified the extra cost.

Rejected — finer decomposition (splitting calibration, TDA, and scenario execution into their own services). Maximum isolation, but it multiplies the network hops inside a single user action, turns one saga into three, and produces services too small to own a meaningful contract. The team's operating capacity (§1) makes this a net loss.

Rejected — coarser grouping (for example folding XAI into ML and Dashboard into the gateway). Fewer moving parts, but it re-merges lifecycles that genuinely differ: XAI's dependency on an external LLM provider and its narrative-generation churn have nothing to do with model training, and giving the gateway domain responsibilities destroys its property of holding no business logic (§4).

4. Logical architecture

Microservices changes what "layered architecture" means, and the change is worth stating plainly because the brainstorm's single six-layer stack — presentation, API, domain services, event backbone, data access, data, each layer calling only the layer below — no longer describes the system, and two of its features were wrong for this design even before the style changed. A system-wide **domain service layer** implies the seven

domains share one deployment and one data path, which is exactly the property A1 gives up. And placing the **event backbone between the domain and the data-access layer** puts a log on the path from a service to its own database: writes would be slower, reads would be indirect, and a broker outage would stop a service from reading data it owns. Kafka belongs to the side of the request path, carrying facts *between* services, not underneath one (§4.1 Tier 4, §7.4).

Layering itself survives — it now exists **inside each service**, repeated eight times, in the form §4.2 fixes. At the system level the picture is not a stack of layers but a **topology of autonomous services** joined by an edge, a bus, and nothing else. Two diagrams are therefore needed instead of one.

4.1 System-level view — five tiers, not six layers

The difference from the brainstorm's stack is not that one layer was dropped: the tiers below are *deployment* tiers describing where code runs and what may address what, whereas the brainstorm's layers were *call* layers inside one application. Tier 4 in particular is not a hop the request path passes through.

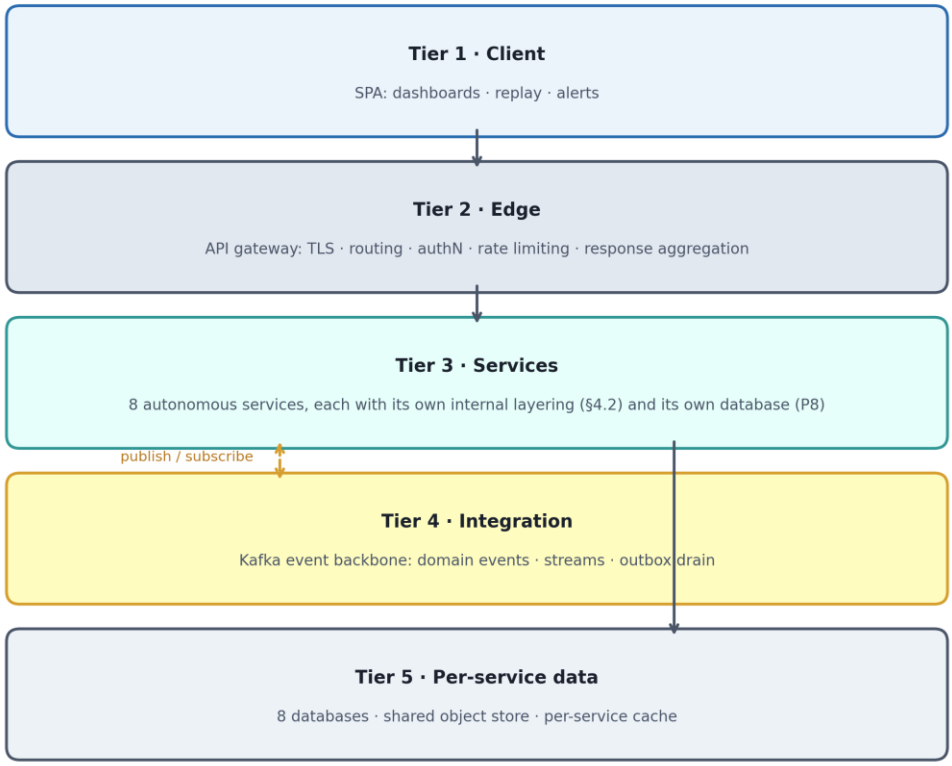


Figure 4.1: System-Level View — Five Deployment Tiers

Tier 1 — Client. A single-page application holding no business logic and no database access. It speaks only to the gateway over HTTPS, and receives pushed alerts and live progress over a WebSocket/SSE channel (§7.3, §9.6). Critically, it talks to **one** origin: the browser never addresses individual services, so the decomposition is invisible to it and can change without a frontend release.

Tier 2 — Edge (API gateway). The single front door and the only component with a public address. Its responsibilities are strictly cross-cutting: terminate TLS, route each path prefix to the owning service, authenticate the caller by verifying the JWT, apply rate limiting, and — where a screen genuinely needs data from several services — aggregate a small number of fan-out reads into one response so the browser is not forced into a chatty conversation. It holds **no business rules and no database**. What the gateway may *not* do is make authorization decisions on a service's behalf: it authenticates and passes a verified identity inward, while each service authorizes the specific action against its own rules (§9.2). This is deliberate defence in depth — a gateway-only check would mean any request that reached a service by another path was implicitly trusted.

Tier 3 — Services. Eight autonomous services (§5). Each owns a bounded context, a database, and a published contract. A service never reads another's database (P8); it either calls that service's API or reacts to its events. This tier is where all business meaning lives.

Tier 4 — Integration (event backbone). Kafka, carrying three kinds of traffic: high-volume streams (ticks, order lifecycle), coarse domain events other services react to, and the drain of each service's transactional outbox (§7.7). It is drawn to the side of the request path deliberately — it is a decoupling and durability mechanism, not a hop every request passes through.

Tier 5 — Per-service data. Eight logically separate databases, a shared object store for large immutable artifacts, and per-service caches. "Shared" for the object store means one *system*, not one namespace: each service writes only under its own prefix with its own credentials, so ownership survives the shared infrastructure (§8.1).

4.2 Service-internal view — the layering, repeated in every service

Each service is internally layered, and because every service uses the same template (§5.1) the layering is identical everywhere — which is what makes eight services learnable by three people.

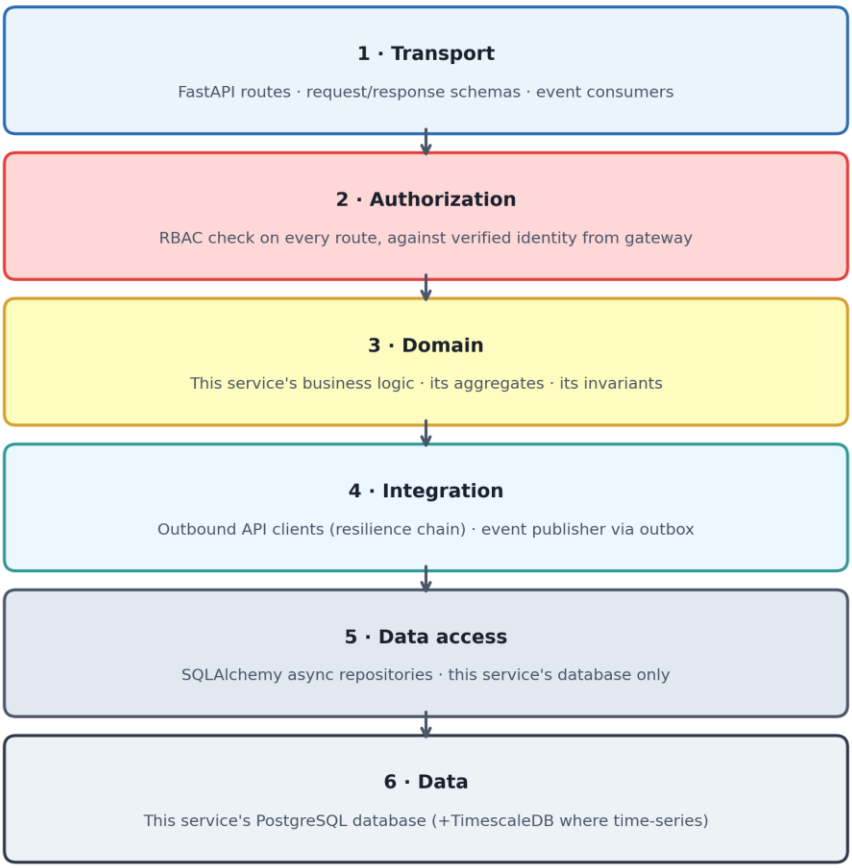


Figure 4.2: Service-Internal View — Layering Repeated Across Services

The rule inside a service is the familiar one — a layer calls only the layer beneath it — with one addition that distributed operation forces: **all outbound communication is confined to layer 4**. Domain code never opens an HTTP connection and never publishes to Kafka directly; it records an intent and layer 4 turns it into a resilient call or an outbox row. This keeps the resilience policy (§7.5) in one place per service instead of scattered through business logic, and it is what makes a service testable without the rest of the platform running.

5. Service architecture

Each service is an autonomous unit: it owns a bounded context, a database, a published contract, a deployment pipeline, and its own scaling policy. It interacts with other services only through their published APIs or their published events (P8). This section fixes what each service owns and what it exposes; the module numbering (M1–M7) from the proposal is retained in the headings so the proposal, this document, and the requirements specification all use one vocabulary.

5.1 The shared service template

Eight services built eight different ways would be unmaintainable by three people, so every service is generated from **one template** and differs only in its domain layer. The template fixes the internal layering

(§4.2), the framework (FastAPI + async SQLAlchemy), the health and readiness endpoints Kubernetes probes, structured logging with correlation-id propagation, OpenTelemetry trace instrumentation, the outbox table and its drain worker (§7.7), the resilient outbound HTTP client with its timeout/retry/circuit-breaker policy (§7.5), the Kafka consumer wrapper with idempotency and offset handling, the RBAC dependency that every route declares (§9.2), the Alembic migration setup, and the Dockerfile and CI pipeline shape.

The payoff is concrete: a developer who has worked in one service can navigate any other, cross-cutting fixes are made once in the template rather than eight times, and the *interesting* differences between services — the mathematics, the data model, the failure behaviour — are the only differences there are. This is the primary mitigation for the operational-surface cost accepted in A1.

5.2 Service inventory

#	Service	Owns database	Compute profile	Deployments	Publishes	Depends on
0	**Platform / Identity** (M0)	`platform_db`	I/O-bound, light	API	`user.deactivated`, `role.changed`	— (every service verifies its tokens locally; none calls it per request, §9.1)
1	**Market Data** (M1)	`market_db`	I/O-bound, scheduled	API + scheduled worker	`bar.ingested`, `news.ingested`, `tick.*`, `replay.*`	Platform (identity)
2	**Simulation** (M2)	`simulation_db`	**CPU-bound, bursty**	API + CPU workers	`sim.progress`, `sim.completed`	Market (calibration inputs)
3	**Trading** (M3)	`trading_db`	Event-sequenced, moderate	API + engine worker	`order.*`, `fill.*`, `portfolio.*`	Market, Simulation (market env); ML *optionally*, where a strategy consumes predictions
4	**ML** (M4)	`ml_db`	**CPU/GPU-bound, bursty**	API + GPU/CPU workers	`prediction.ready`, `model.registered`	Market, Trading — **via events only**, never a synchronous call (§8.6)
5	**Risk & TDA** (M5)	`risk_db`	CPU-bound (homology)	API + CPU workers	`stress.completed`, `regime.transition`, `risk.threshold.breached`	Market, Trading, Simulation
6	**Explainability** (M6)	`xai_db`	I/O-bound (LLM calls)	API + worker	`explanation.ready`	ML, Risk, external LLM
7	**Dashboard &**	`dashboard_`	I/O-bound,	API + alert-	`alert.trigger`	subscribes to all;

#	Service	Owns database	Compute profile	Deployments	Publishes	Depends on
	Alerting (M7)	db	read-heavy	evaluator worker	ed	queries services for detail
—	**API Gateway**	*(none)*	I/O-bound	API	—	routes to all services

A service's **Depends on** column is a statement about its *runtime* dependencies, and every one of them is required to degrade rather than cascade (P9). The dependency graph is deliberately shallow and **acyclic on the synchronous path**, which the Trading↔ML pair is the reason for spelling out: Trading may read ML's latest prediction synchronously when a strategy is configured to use one, with a declared fallback to rule-based logic if ML is unavailable (§11.1), while ML's dependence on Trading runs the other way entirely — it materializes portfolio and fill features from Trading's *events* (§8.6). Because one direction is a call and the other is a subscription, there is no synchronous cycle to deadlock, and neither service has to be up for the other to start.

Deployment count, stated once to avoid ambiguity later. Nine *deployable units* means nine independently releasable components: eight services plus the gateway. Those units expand into more Kubernetes Deployments than that, because seven of the eight services ship a worker alongside their API — Market Data a scheduler, Simulation and Risk CPU workers, ML a GPU worker, Trading its ordering engine, XAI its LLM worker, Dashboard its alert evaluator — and only Platform/Identity is API-only. So the platform runs **eight service APIs, seven workers, and one gateway: sixteen Deployments from nine releasable units**, and each worker scales on its own axis independently of its own API (§10.2, §10.3). Where this document says "unit" it means the releasable component; where it says "deployment" it means the running Kubernetes object.

5.3 Platform / Identity service (M0)

Owns accounts, the fixed role set, sessions and refresh tokens, row-scope grants (which portfolios and instruments a user may see), and the **audit sink**. It issues the JWTs that every other service verifies, and it is the one service every other service depends on, which makes its availability requirement the strictest in the system — it is stateless apart from its database and therefore the easiest to replicate.

Audit deserves a specific note, because microservices changes where it can live. Each service audits *its own* writes into *its own* database using the generic trigger (§8.7) — a service cannot write to another's tables, so a single central audit table written directly by triggers everywhere is impossible. Platform additionally consumes an `audit.recorded` event stream so it can answer cross-service governance questions ("what did this user change anywhere, in order?") without becoming the write path for audit. Local capture stays authoritative and synchronous; the aggregated view is eventually consistent and is explicitly *not* the compliance record.

5.4 Market Data service (M1)

Acquires OHLCV bars, indices, equities, and financial news; cleans and normalizes them; computes and **stores** returns, volatility, and correlation matrices once so downstream services never recompute them (P6); links each news article to the instruments it references; and owns the historical **replay session** abstraction (create,

play, pause, control speed). It realizes P1's market environment: a replay session and a simulation run both expose the same *market-environment* contract that Trading binds to, and under microservices that contract is now an explicit published interface plus a tick event stream rather than an in-process abstraction. Ingestion runs mostly on a schedule, so its heavy path is a scheduled worker; each run records status, row count, and errors, and re-running a window must not double-insert.

Cleaning is a gate, not a post-processing step (FR-MKT-011). Nothing reaches the published tables or the `bar.ingested` stream until it has passed the cleaning stage, because a distributed system cannot retract a bad bar cheaply: once a defective price is published, every downstream read model has already projected it and every feature computed from it is silently wrong. So the pipeline lands raw payloads in a staging table, then detects and records missing bars against the instrument's trading calendar, duplicate bars, non-positive or out-of-range prices, and zero-volume anomalies; aligns every timestamp to UTC; and normalizes news text for encoding, duplication, and boilerplate. Each run emits a **reconciliation report** — what it corrected, what it rejected, what passed untouched — stored alongside the run row so a suspicious downstream number can always be traced back to the state of its inputs. Rejected rows are retained rather than dropped, since the same defect appearing every week is a vendor problem worth seeing.

Rolling correlation belongs here, not in Risk. The TDA pipeline consumes a *series* of correlation matrices over sliding windows (§8.10), and the tempting shortcut is to let Risk compute them from bars it fetches. This design rejects that, for three reasons: a correlation matrix is derived market data by definition, so putting it anywhere else splits one bounded context across two services; ML wants the same matrices as features, and two independent implementations of the same rolling window would eventually disagree; and a matrix over a *closed* historical window is immutable, so computing it once and caching it is strictly cheaper than recomputing it per consumer (P6). Market Data therefore exposes the rolling series as a **single batched call** — a basket, a window length, and a step, returning every matrix in the range (FR-MKT-004) — rather than forcing a consumer analysing two hundred consecutive windows to make two hundred requests. Each matrix carries a content fingerprint so Risk can record exactly which inputs a given topological result was derived from.

Market Data is the most widely depended-upon domain service, so its read API is aggressively cacheable (bars and correlation matrices for a closed historical window are immutable, and are served with long-lived cache headers and ETags) — which is what stops four services' feature computations from hammering it.

5.5 Simulation service (M2)

Calibrates GBM, Ornstein–Uhlenbeck, jump-diffusion, and regime-switching models to a chosen instrument over a chosen window, stores the fitted parameters keyed by calibration type, and generates Monte Carlo paths (N paths × T steps) plus precomputed run statistics (mean, VaR, CVaR). Its API validates and enqueues a run; a CPU worker executes it, publishes progress, writes paths and stats, and publishes `sim.completed`.

Two consequences of the service split are worth recording. First, calibration needs price history that Market Data owns, so Simulation fetches it over the API at calibration time and **stores the input window's fingerprint** (instrument, range, row count, hash) alongside the fitted parameters — so a calibration remains reproducible and auditable even though the source data lives in another service. Second, the instrument-match invariant (a run's instrument must equal its calibration's instrument) is enforceable in `simulation_db` because both rows live there (§8.9); but the instrument's *existence* is Market Data's fact, so Simulation validates it via the API on write and keeps a local instrument reference projection for display (§8.12). This is

the general pattern for every "foreign key that crosses a service boundary": validate on write, project for read, never join.

5.6 Trading service (M3)

Binds a portfolio and strategy to a market environment and processes an ordered stream of events — tick arrives → strategy evaluates → order placed → matched/filled with transaction cost and slippage applied → portfolio updated — in strict chronological order. Its defining internal property is **event sourcing**: the immutable, per-portfolio, strictly-ordered event log in `trading_db` is the source of truth, and the portfolio's value series, cash series, and holdings history are append-only *projections* derived from it (P3, §8.5). Order types supported: market, limit, stop, stop-limit.

Trading is the service where the distributed design must be most careful, because it is the only one holding a strict total order. It consumes the tick stream from Market Data (or simulated paths from Simulation) as a **partitioned, keyed Kafka stream**, and consumes it with a single consumer per portfolio partition so events for one portfolio are never processed concurrently or out of order. Its consumers are idempotent on `(portfolio_id, seq)`, so at-least-once redelivery cannot duplicate an event in the log. This is why the engine is a worker deployment rather than a horizontally scaled API: correctness here is a function of ordering, and ordering is a function of partition ownership.

5.7 ML service (M4)

Produces sentiment scores, extracted financial events, regime classifications, volatility forecasts, drawdown-risk probabilities, portfolio health scores, and strategy metrics. Two internals dominate its data model. First, a **model registry** (task type, version, hyperparameters, artifact reference path, training-run metadata) so every prediction is attributable to an exact model version and is reproducible (P5, §8.6). Second, a **point-in-time feature store** whose access pattern makes look-ahead bias structurally impossible: a feature snapshot as-of timestamp t can only read data whose knowledge-time $\leq t$ (§8.6). Training and batch inference run on GPU-capable workers; cheap single-row inference runs on the API tier.

The proposal names four candidate method families, and each maps to a task the service already owns: **XGBoost** and **Random Forest** for the tabular supervised targets (drawdown-risk probability, regime classification, portfolio health scoring) where engineered features and gradient-boosted trees remain the strongest baseline; **LSTM** for sequence targets such as volatility forecasting, where the ordering of the input carries the signal; and **transformer-based financial NLP** for sentiment scoring and event extraction from news text. The registry is what keeps this open rather than committed: because a model row records its task type, its family, and its hyperparameters, adding or replacing a family is a registration plus a training run, not a redesign — so the service is architected to hold several families per task and compare them, and no ADR in §13 fixes a single winner in advance. Only the transformer path needs a GPU as a practical matter, which is why the GPU node selector applies to the ML worker and to nothing else in the platform (§10.2).

The feature store is the sharpest illustration of P8's cost and why it is still worth paying. ML's features derive from data owned by Market Data and Trading, and ML may not read those databases. So ML **materializes its features into its own store from those services' events**, stamping each value with the `knowledge_time` at which the event was received. This is more work than a join — and it is strictly better for correctness, because the knowledge-time recorded is the time the fact actually became available to the model, which is

exactly the quantity look-ahead-bias prevention needs. A join against another service's current table would have silently used data revised after the fact.

5.8 Risk & TDA service (M5)

Two capabilities behind one database. **Stress testing** injects configured scenarios (flash crash, liquidity shock, volatility spike, sector-wide collapse, correlated-asset failure) against a portfolio, optionally on top of a synthetic market from Simulation, and reports max drawdown, VaR, recovery days, the post-shock recovery trajectory, and loss attribution by instrument, sector, and geography. It also reports **risk concentration** — largest single-instrument and largest sector share of exposure, plus a concentration index over the exposure distribution — computed both for the portfolio as it stands and as it behaves under a stress run (FR-RSK-018); concentration is measured here rather than in Trading because it is a risk statistic over an exposure snapshot, and pairing the two readings is what turns "this portfolio is concentrated" into "this is what that concentration costs under a flash crash". **TDA** runs the pipeline — rolling correlation → distance matrix → Vietoris–Rips complex → persistent homology → Betti numbers → persistence landscape → composite topological risk score — over sliding windows, and flags structural regime transitions between consecutive windows. The first stage is a *fetch*, not a computation: the rolling correlation matrices are Market Data's output, obtained in one batched call and recorded by fingerprint (§5.4, §8.10), and Risk owns the pipeline from the distance matrix onward. Both capabilities are compute jobs on CPU workers. The scenario catalog is registry-driven so a new scenario is a row, not a migration (P5).

This service is the strongest justification for the microservices decision in practice: persistent-homology computation is CPU-hungry and its libraries (Ripser, GUDHI) are heavyweight native dependencies that nothing else in the platform needs, and it is the most research-volatile code in the system. As its own service it scales on its own nodes, ships its own image, and can be redeployed mid-experiment without touching trade execution.

5.9 Explainability service (M6)

Consumes predictions and risk results and explains them: SHAP feature attributions for any model prediction, asset-level contribution to portfolio P&L, risk decomposition by factor, event-to-impact links, and plain-English narration via an external LLM API. Its interactions are mostly reactive — it subscribes to `prediction.ready` and `stress.completed` — and its only slow dependency is the LLM call, which runs on a worker with a timeout, a circuit breaker, and a cached-or-templated fallback so an unavailable provider degrades narration rather than failing the request.

SHAP attribution needs the model artifact and the exact feature vector, both owned by ML. Rather than reaching into `ml_db`, XAI requests an **explanation bundle** from ML's API — the artifact reference plus the frozen feature vector for that prediction — and caches it. This keeps the boundary intact and has the useful side effect that an explanation is computed against precisely the inputs the prediction used.

5.10 Dashboard & Alerting service (M7)

Serves the read models for every widget (portfolio overview, trade replay, risk heatmaps, volatility, drawdown, stress results, strategy comparison, regime and topological-risk views), persists per-user layouts and widget configuration, evaluates user-defined alert rules against the domain-event stream, and records

every alert trigger including acknowledgement. Alert delivery is **push** (server → browser over WebSocket/SSE), never poll (§9.6).

Dashboard is a **read-model service**, and this is the design decision that makes the dashboard latency target survivable under microservices. If each widget fanned out to live calls across five services, a page load would multiply latency and couple its availability to every service at once. Instead Dashboard subscribes to the domain events of every service and maintains its own denormalized, query-shaped projections in `dashboard_db` (§8.12). A widget read therefore touches one database — its own. The cost is bounded staleness, measured in the lag between an event and its projection, which is acceptable for exactly the reason the read replica was acceptable in version 0.1: these are analytical views that were already reading precomputed aggregates. Where a user drills into live detail, Dashboard calls the owning service directly and degrades that panel alone if the call fails.

Alert evaluation runs in Dashboard's own worker, and its hard problem is duplicates rather than detection (FR-DSH-009). The evaluator is a continuous consumer (§6): it reads the domain-event stream, matches each event against the user-defined rules stored in `dashboard_db`, and writes a trigger row. Detection is easy; not alarming the user five times about one condition is not, because at-least-once delivery means the same event can arrive more than once, a redeployed evaluator may replay from its last committed offset, and a metric hovering on a threshold will legitimately cross it repeatedly. Three mechanisms handle that. Each trigger row carries a **deterministic key** (rule id + the identity of the evaluated subject + the window or event id that satisfied it) under a unique constraint, so a replayed event updates a row instead of inserting a second one. Each rule holds **state, not just a predicate** — a rule that has fired stays `firing` and does not re-notify until it has been observed in the clear, so a value oscillating around its limit produces one alert and one resolution rather than a stream. And each rule carries a **cooldown** after which a still-true condition may re-notify, so a genuinely persistent problem is not silently forgotten either. Notification is a side effect of the row transitioning to `firing`, never of the event arriving, which is what keeps delivery idempotent end to end.

6. Execution & concurrency model

The line between the interactive plane and the compute plane (P2) is implemented as a **job** abstraction, and under microservices that abstraction is **local to each compute-heavy service** rather than platform-wide. Simulation, ML, Risk, and XAI each own a `job` table in their own database and their own queue; each validates and enqueues on its API tier and executes on its own worker deployment. There is no shared job service and no shared worker fleet, because a shared fleet would reintroduce exactly the coupling the decomposition exists to remove: one service's queue backlog would delay another's work, and the pool would have to hold every service's dependencies in one image.

Not every worker is a job runner, and the distinction matters for how each one is monitored and scaled. Four of the seven are **job workers** in the sense above — a user or an upstream event submits a unit of work, its row moves through `queued → running → succeeded|failed`, and the caller polls or waits for a completion event. The other three are **continuous workers** with no user-submitted job: Market Data's scheduler wakes on a cron-like interval to ingest, Trading's engine consumes its order stream in sequence and never idles between explicit submissions, and Dashboard's alert evaluator runs a standing evaluation loop over thresholds. Job workers are measured by queue depth and job latency and scale on backlog; continuous workers are measured by consumer lag and are scaled by partition count instead (§10.3), which is why the two kinds are never merged into one deployment even where the code would fit.

The lifecycle within a service is unchanged in shape: the API validates the request, writes a job row in status `queued`, enqueues its id, and returns immediately with the job id. A worker claims the job, sets it `running`, publishes progress events, writes results to its own database and to the object store, sets it `succeeded` or `failed`, and publishes a completion event that other services — Dashboard for a live update, XAI to begin an explanation — react to.

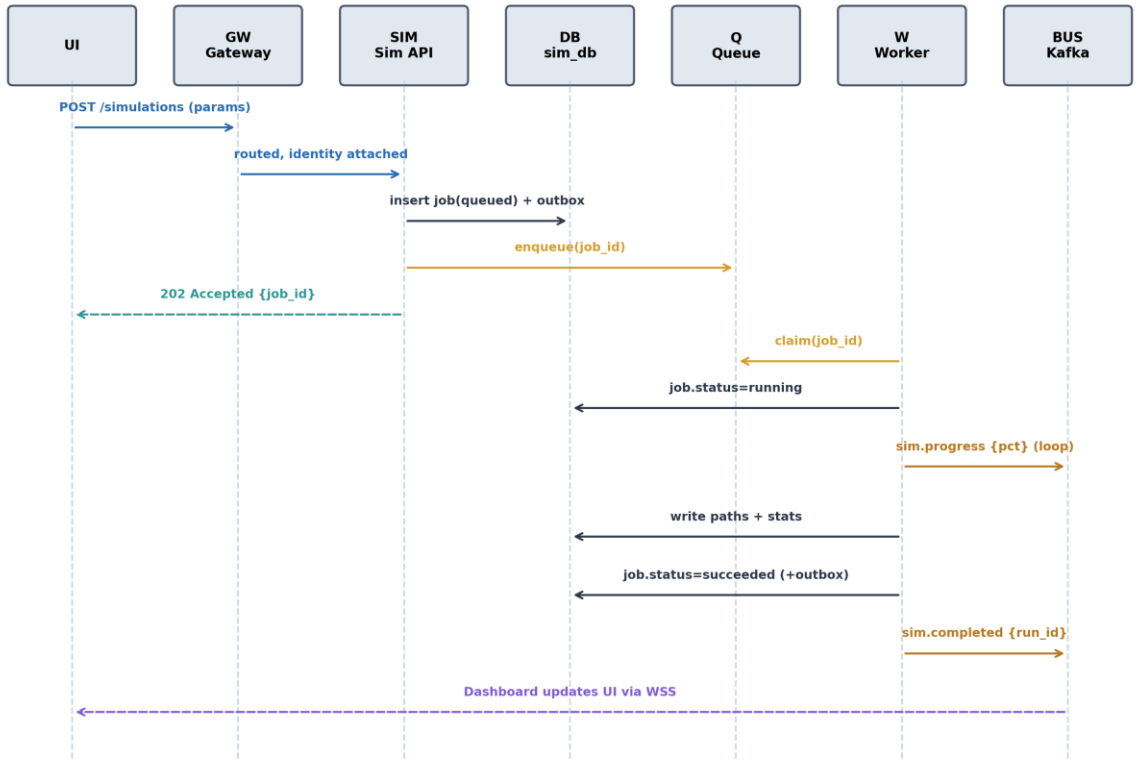


Figure 6.1: Asynchronous Execution Sequence — Simulation Job Lifecycle

Note what the diagram makes explicit: the browser's progress updates arrive via **Dashboard**, not via Simulation. Simulation publishes facts and knows nothing about who is watching; Dashboard owns the client connection and the push channel (§9.6). This keeps the WebSocket fan-out concern in one service and means a Simulation redeploy does not drop every client's live connection.

Worker deployments are per-service and sized to their own scarce resource. Simulation and Risk workers are CPU-bound and scale on core count and replica count. ML workers are GPU-capable, and GPU jobs are serialized against a bounded device budget inside the ML service so two trainings do not thrash one device — a constraint that is now genuinely ML's own business rather than a platform-wide scheduling problem. Each service therefore has an independent scaling knob (§10.3), which is the concrete mechanism behind "compute throughput scales without degrading interactive latency": the workers are not merely different processes, they are different deployments with their own resource limits.

Monte Carlo is embarrassingly parallel and is exploited as such: a run of *N* paths is partitioned across the Simulation service's worker cores and replicas, each producing a slice of paths written in bulk. Because paths are independent, throughput scales close to linearly, and a large run degrades gracefully into more wall-clock time rather than a failure.

Jobs are idempotent and durable. A job row carries a unique idempotency key derived from its parameters, so a retried or duplicated submission cannot produce two result sets; a worker that dies mid-job leaves the job reclaimable through a lease/visibility timeout. This matters more under microservices than it did in a monolith, because a caller that times out on a job submission genuinely cannot tell whether the job was accepted, and its retry must be safe. The same reasoning applies to event consumers, which are idempotent by construction (§7.5).

Backpressure is explicit and per-service. Each service's queue depth is a first-class signal: when its compute plane is saturated, jobs wait in `queued` and the UI shows position and estimated wait, rather than the API attempting the work and timing out. Because the queues are separate, a saturated Risk service does not slow simulation submissions — the isolation is real, not nominal.

7. Communication & integration architecture

QuantSim uses four distinct communication mechanisms, and keeping them distinct is itself a design decision — collapsing them (as an early draft that "puts everything on Kafka" would) produces either a job queue with no retry semantics or an event log abused as an RPC channel. The governing rule for choosing between them is: **synchronous when the caller cannot proceed without the answer and the answer is a read; asynchronous for everything that changes state across a boundary.**

7.1 Synchronous request/response (client → gateway → service, and service → service). Ordinary reads, CRUD, and job submission arrive as HTTP/JSON at the gateway, which routes by path prefix to the owning service. Service-to-service calls — Simulation fetching a price window from Market Data, XAI requesting an explanation bundle from ML — use HTTP/JSON as well, with **gRPC reserved for the two paths where payload size and serialization cost justify it** (bulk price windows and feature vectors), because gRPC's protobuf contracts and streaming are worth the extra toolchain only where the volume is real.

Every service publishes an **OpenAPI contract** generated from its FastAPI route models, and contracts are versioned in the URL path (`/v1/...`). The compatibility rule is explicit: additive changes (new optional field, new endpoint) ship freely; any breaking change requires a new version path served alongside the old until consumers migrate. Consumers are protected by **contract tests** — each consumer contributes a test asserting the shape it depends on, and those tests run in the provider's pipeline, so a provider learns it has broken a consumer before deploying rather than after. This is the discipline that makes independent deployability real; without it, "deploy independently" means "break each other independently."

Synchronous calls are used sparingly and never chained more than one hop deep. A request that would require a chain across three services is a design smell that gets resolved by projecting data locally (§8.12) instead of calling further.

7.2 Per-service task queue (service API → its own workers). Job dispatch (§6) uses a task queue with claim/lease/retry/dead-letter semantics. The implementation is an async-native Redis-backed queue (**Arq**, or **Celery** where its tooling is preferred), and each compute-heavy service runs its **own** queue namespace against its own Redis logical database. The queue is *not* the event backbone: job semantics (claim, lease, retry, dead-letter, single consumer) and log semantics (durable ordered replay, many independent consumers, offsets) are different problems, and conflating them loses one of the two.

7.3 Live server-to-client push (Dashboard → browser). Simulation and replay progress, and alert notifications, are pushed to the browser over **WebSocket** (bi-directional, used for interactive replay controls)

or **SSE** (one-way, sufficient for alerts). The **Dashboard service owns every client connection**; other services publish facts to Kafka and Dashboard translates them into pushes (§6). Concentrating connection state in one service means only one service needs sticky routing and connection-count scaling, and no other service's deployment disturbs a live client session.

7.4 Event backbone (service ↔ service, async). Kafka carries all cross-service state propagation. Three event families ride it:

- *Streaming events* — market ticks and order-lifecycle events during a run. High-volume and ordered per key (per instrument, per portfolio), which maps directly onto Kafka partitioning by key and is what allows Trading's strict ordering guarantee (§5.6).
- *Domain events* — coarse-grained facts other services react to: `sim.completed`, `stress.completed`, `regime.transition`, `prediction.ready`, `risk.threshold.breached`. These decouple producers from consumers: Simulation does not know Dashboard exists; it announces completion and Dashboard reacts.
- *Read-model feeds* — the same domain events consumed by services maintaining local projections (§8.12), including ML's feature materialization and Dashboard's widget models.

Delivery guarantees and schema discipline. Production runs replication factor ≥ 3 with `min.insync.replicas = 2` and producer `acks=all`, so a single broker loss causes no event loss; development runs a single broker with the same producer settings so code paths are identical (A2). Delivery is **at-least-once**, and every consumer is therefore idempotent — keyed on a natural business key or a deduplication table of processed event ids. Event payloads carry an explicit `schema_version` and are registered as JSON Schema (Avro where a topic's volume justifies it) in a schema registry, with the same additive-compatibility rule as the HTTP contracts. Consumers ignore unknown fields, which is what lets a producer add a field without coordinating a release across eight services.

7.5 Resilience on every hop. Because the network is an ordinary failure condition (P9), every outbound call from layer 4 of the service template (§4.2) is wrapped in a uniform policy, and the policy is in the template rather than in each developer's judgement:

Mechanism	Setting	What it prevents
Timeout	Per-call budget, always set; default 2 s for reads, 5 s for bulk	A slow dependency consuming the caller's capacity indefinitely
Retry	Bounded (2 attempts), exponential backoff **with jitter**, only for idempotent operations	Transient loss becoming a user-visible failure; jitter prevents synchronized retry storms
Circuit breaker	Opens after a failure-rate threshold, half-opens to probe	Hammering a service that is already down, and turning its outage into the caller's outage
Bulkhead	Bounded concurrency per dependency	One failing dependency exhausting the caller's connection pool and taking down unrelated paths
Fallback	Explicit per call site: stale cache, queued work, or degraded response	A dependency failure propagating as a 500 instead of a reduced feature

The rule that makes this coherent is that **every call site must name its fallback**. A dependency being unavailable is never allowed to mean "hang" or "unhandled exception"; it means a specifically-chosen degradation, recorded in the service's own documentation and asserted in its tests. §11 lists the degradations for each dependency.

7.6 Cross-service consistency: choreographed sagas. With one database per service there are no distributed transactions, so any workflow that changes state in two services is a **saga**: a sequence of local transactions, each publishing an event that triggers the next, with an explicit **compensating action** for each step that has already committed. Choreography (services reacting to events) is chosen over an orchestrator because the workflows here are short and the coordinator would become a coupling point that every service depends on.

The load-bearing example is *run a stress test against a portfolio on a synthetic market*: Risk records a stress run as `pending`; it needs the portfolio's positions from Trading and a simulated market from Simulation. Rather than a transaction spanning three databases, Risk commits its own `pending` row, requests the inputs, and advances its own state as `sim.completed` and the position snapshot arrive; if Simulation's run fails, Risk's compensating action marks the stress run `failed` with a reason and emits `stress.failed`, leaving no partial result visible. The invariant preserved is not "all three databases changed atomically" — that is unachievable and is not required — but "**no service ever exposes a state that implies another service's work succeeded when it did not**." Every saga in the system is documented as a state machine with its compensations, and the state a user sees is always a committed local state, never an inferred one.

7.7 The transactional outbox. The subtlest failure in an event-driven system is the gap between "I committed my state" and "I published my event": if a service commits and then crashes before publishing, the rest of the platform never learns, and the databases diverge permanently with nothing to detect it. Two-phase commit across Postgres and Kafka is not available, so every service uses the **outbox pattern**.

```
-- present in every service database, provided by the shared template (§5.1)
CREATE TABLE outbox_event (
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  topic          text        NOT NULL,
  event_key      text        NOT NULL,       -- Kafka partition key (e.g. portfolio_id)
  schema_version int         NOT NULL,
  payload        jsonb       NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now(),
  published_at   timestamptz                 -- NULL until the drain confirms the broker ack
);
CREATE INDEX ON outbox_event (id) WHERE published_at IS NULL;  -- the drain's hot path
```

The service writes its business rows **and** its outbox row in **one local transaction**, so either both happen or neither does. A small drain worker in each service then reads unpublished rows in id order, publishes to Kafka, and marks them published only after the broker acknowledges. A crash between publish and mark causes a duplicate, never a loss — which is precisely why every consumer is idempotent (§7.4). This converts the hard problem (atomicity across two systems) into an easy one (tolerating duplicates), and it is the single mechanism that makes eventual consistency across eight databases trustworthy rather than hopeful.

7.8 Service discovery and service-to-service authentication. Services address each other by stable DNS names resolved by the platform — Kubernetes `Service` records in production, Compose network aliases in development — so no service holds another's address in configuration and no separate discovery registry is operated. Calls between services carry a **short-lived service credential** issued to the calling workload, distinct

from the end-user JWT; when a call is made on a user's behalf the user identity is forwarded alongside it, so the receiving service authorizes the *user's* action rather than trusting the caller (§9.2). Internal endpoints are not routable from outside the cluster: the gateway is the only ingress.

8. Data architecture (physical design)

This is the heart of the physical design. It defines where data rests, how it is partitioned and indexed, and how the domain's correctness invariants are enforced in storage.

8.1 Storage tiers

QuantSim uses a three-tier storage strategy because its data has three sharply different access patterns. Under microservices each tier is **partitioned by owning service**, so "shared infrastructure" never means "shared data".

Hot — PostgreSQL 16 + TimescaleDB, one database per service. All operational and time-series data that is queried interactively: instruments, portfolios, orders, the trade-event log, portfolio value series, recent bars, simulation stats, predictions, risk results, TDA outputs. PostgreSQL gives transactional integrity, rich constraints (which P4 relies on), and JSONB (which P5 relies on); TimescaleDB adds automatic time-partitioning, columnar compression, and continuous aggregates for the time-series majority of the data. Which service holds what is fixed in §8.2.

Warm — Redis, one logical database per service. Cache of precomputed payloads and expensive query results, ephemeral job and progress state, and each compute service's task-queue backend (§7.2). Services do not share cache keyspaces — a cached value is derived from a service's own data and is that service's to invalidate. Note the change from version 0.1: Redis is **no longer** the local event backbone, because Kafka now runs in every environment (A2, §7.4).

Cold — one object store, per-service prefixes. Large, rarely-read, append-once data: full Monte Carlo path sets after a run is closed (Simulation), historical raw bars beyond the interactive window (Market Data), and ML model artifacts (ML). Columnar Parquet in S3-compatible storage (MinIO locally) is an order of magnitude cheaper per byte than keeping everything in Postgres and is the natural format for the analytical reads that touch it. One storage *system* is shared, but each service writes only under its own prefix with its own credentials, and a service that needs another's artifact receives a **reference from the owning service's API** rather than reading the other's prefix directly — so ownership survives the shared infrastructure. Each service's data-access layer abstracts tier placement, so its domain code asks for "the paths of run X" without knowing whether they are still hot or have aged to Parquet.

8.2 Database per service

Data is divided into eight physically separate databases, one per service, and no service holds credentials to any database but its own (P8). This is the decision that distinguishes microservices from a modular monolith at the data layer, and it is the one with the widest consequences in this document: there are no cross-service foreign keys, no cross-service joins, no shared tables, and no shared `audit` schema. Version 0.1 of this document used eight *schemas* in one database; that arrangement is withdrawn, because a shared instance leaves every service one convenient `JOIN` away from a distributed monolith, and shares a failure domain besides.

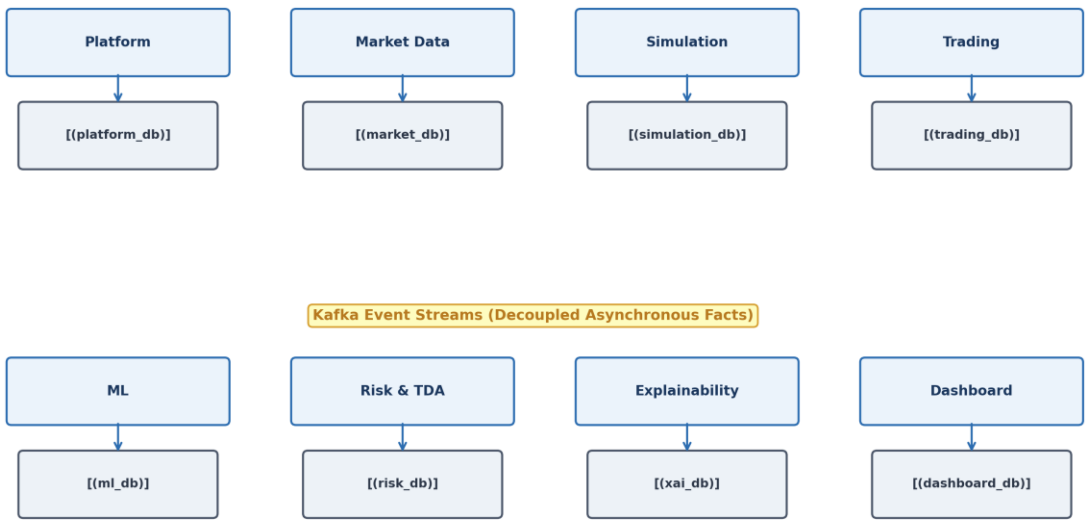


Figure 8.1: Database per Service Ownership and Asynchronous Event Flows

The dotted arrows are **event flows, not foreign keys** — this is the critical difference from the relational diagram version 0.1 carried. Where a downstream service needs upstream data on its read path, it consumes those events and maintains its own local projection (§8.12); where it needs a fact only at write time, it validates over the API and stores a copy of what it validated.

Database	Owning service	Core content	Time-series extension
`platform_db`	Platform / Identity	users, roles, sessions, refresh tokens, instrument- and portfolio-scope grants, aggregated cross-service audit view	—
`market_db`	Market Data	instruments, baskets + membership history, OHLCV bars, corporate actions, news, news-instrument links, returns, volatility, correlation matrices, replay sessions, ingestion runs	**TimescaleDB**
`simulation_db`	Simulation	calibrations (+ input-window fingerprints), simulation runs, simulation paths, run statistics, jobs	**TimescaleDB**
`trading_db`	Trading	strategies, portfolios, **trade-event log**, orders, fills, holdings history, portfolio value series	**TimescaleDB**
`ml_db`	ML	model registry, model versions, training runs, **bi-temporal feature store**, predictions, extracted events, jobs	**TimescaleDB** (features)

Database	Owning service	Core content	Time-series extension
`risk_db`	Risk & TDA	stress scenarios, stress runs, results + attributions, TDA windows, distance matrices, persistence diagrams, Betti sequences, landscapes, topological risk scores, jobs	—
`xai_db`	Explainability	SHAP attributions, contribution analyses, risk decompositions, event-impact links, NL explanations, explanation cache	—
`dashboard_db`	Dashboard & Alerting	layouts, widget configs, alert rules, alert-trigger log, **read-model projections** (§8.12)	**TimescaleDB** (projections)

Every database additionally contains the two tables the shared template provides: `outbox_event` (§7.7) and a `processed_event` deduplication table for idempotent consumption, plus its own `audit.change_log` for its own writes (§8.7). Each has its own Alembic migration history, versioned in that service's repository and applied by that service's pipeline — so a schema change is a single-service deployment, which is precisely the independent-deployability property A1 is buying.

Development co-location, stated honestly. Per A2, the eight databases run as eight logical databases inside one PostgreSQL container locally, each with its own role and its own credentials. The ownership rule is preserved by privilege, not merely by convention — `market_svc` has no grant on `trading_db` — so code that violates P8 fails locally exactly as it would in production. What is *not* preserved locally is the isolation of failure domains and of resource contention; that difference is a known and accepted property of the development profile, not a design position.

8.3 Conventions applied to every table

Every business table carries surrogate `id` (UUID v7 — time-ordered, so it is index-friendly like a sequence but globally unique and safe to expose), `created_at`/`updated_at` (UTC `timestamptz`), and, where a row is mutable, an audit trigger (§8.7). Timestamps are always `timestamptz` in UTC; local time is a presentation concern only. Enumerated domains that are *closed and fixed* (like user role) are PostgreSQL `ENUM` types; enumerated domains that must stay *open* (model task type, stress-scenario type) are registry rows, never enums (P5).

8.4 Time-series design (TimescaleDB)

The append-heavy time series — OHLCV bars, simulation paths, trade events, portfolio value series, tick streams — are **hypertables**: tables that Timescale transparently partitions into time-based chunks. This is the single most consequential physical choice for performance, because it turns "scan the whole table" into "scan the relevant chunk."

Chunking. Bars and portfolio-value series chunk by time (e.g. 7-day intervals); simulation paths, which are bursty and keyed by run, chunk by time within a run and are additionally space-partitioned by `run_id` so one run's data is co-located. **Compression.** Chunks older than the interactive window are compressed with Timescale native columnar compression (large ratios on monotonic financial series), directly serving the "cold

data is cheap" goal before data even leaves Postgres for Parquet. **Continuous aggregates.** Rolled-up views the dashboard needs — daily OHLC from intraday bars, portfolio value down-sampled for charting, rolling volatility — are maintained as Timescale continuous aggregates, refreshed incrementally on write. The dashboard reads the aggregate, not the raw series (P6), which is a primary mechanism behind the sub-two-second read target. **Retention.** A retention policy drops or archives raw chunks past their hot horizon after the continuous aggregate and the Parquet copy exist, so hot storage stays bounded.

8.5 Event sourcing for the trading domain

`trading_db` — owned exclusively by the Trading service — is designed around an **append-only event log as the source of truth** (P3), because the requirement to replay a portfolio's entire history exactly is not achievable by storing only current state. The tables below live in a `trading` schema inside that database.

```
-- trading.trade_event : immutable, strictly ordered per portfolio (the source of truth)
CREATE TABLE trading.trade_event (
  portfolio_id  uuid        NOT NULL REFERENCES trading.portfolio(id),
  seq           bigint      NOT NULL,           -- monotonic per portfolio; gap-free
  event_time    timestamptz NOT NULL,           -- simulated market time
  event_type    text        NOT NULL,           -- ORDER_PLACED|FILLED|CANCELLED|TICK|...
  payload       jsonb       NOT NULL,           -- typed per event_type
  recorded_at   timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (portfolio_id, seq)
);
-- hypertable on event_time, space-partitioned by portfolio_id
SELECT create_hypertable('trading.trade_event','event_time',
                         partitioning_column=>'portfolio_id', number_partitions=>8);
-- append-only: no UPDATE/DELETE granted; enforced by role privileges + a guard trigger
```

Current state is never stored authoritatively — it is a **projection** of this log: `trading.holding_history` (non-destructive position history), `trading.portfolio_value` (value/cash/P&L time series), and a small `trading.portfolio_state_current` cache for cheap reads. Projections are append-only too and are rebuilt by replaying the log from `seq = 1`. `trading.order` and `trading.fill` record every order and its actual fill price, quantity, and realized cost — these are the concrete records the log's `payload` references. The monotonic gap-free `seq` per portfolio is what guarantees a total order strong enough for *exact* replay.

No other service reads this log. Risk, ML, and Dashboard all need portfolio state, and none of them may query `trading_db` (P8). Trading therefore publishes `portfolio.*` and `fill.*` events carrying the state changes, and exposes an API for point-in-time snapshots (`GET /v1/portfolios/{id}/state?as_of=…`) served from its own projections. This is a strictly better boundary than a shared table would be: the event log's `seq` gives every published event a definitive order, so a consumer can detect a gap and request a resync rather than silently drifting — a guarantee a periodic table read could not provide.

8.6 Look-ahead-safe feature store and model registry

Two tables in `ml_db` carry correctness weight far beyond their size. Both are owned solely by the ML service, and both are populated from *events* rather than from queries against other services' data (§5.7) — which, for the feature store specifically, is what makes its central guarantee stronger rather than weaker.

The **feature store** stores every feature value with two timestamps: the `event_time` it describes and the `knowledge_time` at which that value became known. A training or inference request is always made *as of* a

timestamp *t*, and the DAL exposes features only through an as-of query that filters `knowledge_time <= t`. Look-ahead bias — training on information from the future — is therefore not something the modeller must remember to avoid; it is structurally unavailable, because there is no query path that returns a feature whose knowledge-time is after the snapshot. This bi-temporal design is the optimal defense against the failure mode that most quietly invalidates financial ML.

```
CREATE TABLE ml.feature_value (
  entity_id      uuid        NOT NULL,   -- instrument or portfolio
  feature_name   text        NOT NULL,
  event_time     timestamptz NOT NULL,   -- the time the value refers to
  knowledge_time timestamptz NOT NULL,   -- the time the value became known
  value          double precision NOT NULL,
  PRIMARY KEY (entity_id, feature_name, event_time, knowledge_time)
);  -- as-of read: WHERE knowledge_time <= :t
```

The **model registry** (`ml.model`, `ml.model_version`, `ml.training_run`) records each model's task type, each trained version's hyperparameters and artifact reference path (the binary itself lives in the object store, not the database), and each training run's loss curve, final metrics, and status. Every stored prediction carries the `model_version_id` that produced it, so any output is reproducible and attributable (P5). Task type is a registry value, not an enum, so adding a new ML task needs no migration.

8.7 Audit subsystem

Auditing is **local to each service**: every mutable business table is audited by a generic trigger that writes to an `audit.change_log` table **in the same database**, capturing operation, table, actor (the authenticated user from a session-local setting), timestamp, and the full `before`/`after` row images as JSONB. A single platform-wide audit table written directly by triggers is impossible under P8 — no service can write to another's database — and would be a shared failure domain besides. The trigger and this table are part of the shared service template (§5.1), so all eight services audit identically.

```
CREATE TABLE audit.change_log (
  id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  table_name text NOT NULL, op char(1) NOT NULL,     -- I|U|D
  row_pk     text NOT NULL,
  before     jsonb, after jsonb,                     -- DELETE keeps 'before'
  actor_id   uuid, changed_at timestamptz NOT NULL DEFAULT now()
);  -- append-only; hypertable on changed_at
```

Because the `before` image is captured on delete, the log **remains queryable after the underlying row is gone** and can report exactly what was deleted, and the full change history of any single record is reconstructable by selecting its `row_pk` ordered by `changed_at`. Two design notes make this optimal rather than merely correct: the trigger function is lean (a single JSONB serialization and insert, no secondary lookups) to keep write overhead low, and `audit.change_log` is itself a compressed hypertable so the audit trail does not dominate storage. This subsystem is designed to be enabled table-by-table so its write cost lands only on tables that need it.

Answering cross-service audit questions. "What did this user change, anywhere, in order?" cannot be answered by one query when the trail is distributed across eight databases. Each service therefore also emits

an `audit.recorded` event (through its outbox, so capture and emission cannot diverge) carrying the actor, service, table, operation, and timestamp — but *not* the row images, which stay in the owning service. Platform consumes this stream into a queryable cross-service index and resolves detail by calling the owning service when a specific before/after image is requested. The distinction matters for correctness of the claim being made: the **local trail is the authoritative, synchronously-captured compliance record**, and the aggregated index is an eventually-consistent convenience for investigation. This document does not claim a globally-ordered audit log across services, because with independent clocks and independent transactions that would be a false guarantee; it claims per-service total order plus a correlation id (§9.5) that stitches one user action's causal chain together across services.

8.8 Read/write separation

Read/write separation now operates at two levels, and distinguishing them matters because they solve different problems.

Across services, the separation is architectural. Dashboard is a read-model service (§5.10): it answers widget queries from its own denormalized projections rather than querying the services that own the underlying data. This is CQRS drawn at the service boundary — the write model (Trading's event log, Risk's persistence diagrams) and the read model (Dashboard's query-shaped projections) are different databases owned by different services, kept in sync by events. The benefit is larger than a replica could give: dashboard reads do not merely avoid contending with writes, they do not touch the writing services at all, so analytical read load cannot affect trade-event ingestion even in principle, and a Dashboard outage cannot affect trading.

Within a service, the separation is physical where the load justifies it. A service whose read volume warrants it — Market Data (every service reads bars), Dashboard (every user reads widgets), Trading (replay visualization scans event history) — runs a **streaming-replication read replica** of its own database, and its data-access layer tags repository methods read or write, binding reads to the replica pool and writes to the primary. Services without meaningful read pressure (Simulation, Risk, XAI) run a single instance; adding a replica is a per-service decision made on that service's own metrics, which is one of the concrete freedoms the decomposition buys.

The small replication lag is acceptable for the same reason it was in version 0.1 — these reads are of precomputed, slightly-delayed aggregates by design — and the staleness a user sees is now bounded by two hops (event-to-projection, then primary-to-replica), which §12 records as an explicit quality target rather than an unstated assumption.

8.9 Enforced invariants (constraints, not conventions)

The domain's correctness edges are enforced in the schema so no code path can bypass them (P4):

- **Monetary precision.** All money and prices are `NUMERIC` with explicit scale (internal calculations `NUMERIC(28,12)`, stored prices `NUMERIC(23,8)`), never `float`/`double`. This eliminates the floating-point drift that would otherwise accumulate across a portfolio's thousands of arithmetic operations.
- **Role domain.** The user role column in `platform_db` is a PostgreSQL `ENUM('admin','quant','analyst','viewer')`, so an out-of-set role is rejected by the database, not merely by the application.
- **Temporal validity.** `market.replay_session` (in `market_db`) carries `CHECK (end_time > start_time)`, so an inverted replay window is impossible to persist.

- **Simulation–calibration match.** A `simulation.run` references its `calibration` **within `simulation_db`**, and a composite foreign key plus a `CHECK` ties the run's `instrument_id` to the calibration's `instrument_id`, so a run against a mismatched instrument cannot be stored. This invariant survives the decomposition intact precisely because both rows are owned by the same service — which is evidence the boundary was drawn in the right place.
- **Append-only tables.** Event log, projections, and audit have no `UPDATE`/`DELETE` privilege granted to the application role, backed by a guard trigger — immutability is a database guarantee.

Which invariants survive, and what replaces the ones that do not. Every invariant above is *intra-service*, and that is not a coincidence: a constraint can only be enforced by the database that holds both sides of it. Invariants that would span services — "a portfolio's instrument exists in the instrument master", "a stress run's portfolio exists" — **cannot** be foreign keys any more, and pretending otherwise is the most common way a microservices data model quietly breaks. They are handled by a three-part mechanism instead, and the honest statement is that this is weaker than a foreign key and is accepted as a cost of A1:

First, **validate on write**: the service calls the owning service to confirm the referenced entity exists before committing, and stores a copy of the identifying fields it validated, so its own rows remain self-describing and displayable even if the other service is later unavailable. Second, **react to invalidation**: the owning service publishes events when a referenced entity is retired (`instrument.retired`, `user.deactivated`), and holders react by marking their dependent rows accordingly rather than being left with a dangling reference nobody noticed. Retirement is always non-destructive (§8.3 conventions, FR-MKT-001), which is what makes this tractable — there is no hard delete to leave a hole. Third, **reconcile**: a scheduled per-service consistency check compares its local references against the owning service's current set and reports divergence as a metric, so drift is detected and visible rather than discovered by a user. Detection plus repair replaces prevention; that trade is the essence of distributed data.

8.10 TDA data model

The TDA pipeline's intermediate and final artifacts are persisted in `risk_db` per analysis window so results are comparable across time and cheap to redisplay: `risk.tda_window` (window bounds, basket, parameters), `risk.distance_matrix` (the correlation-derived distances), `risk.persistence_diagram` (birth/death pairs per homology dimension, stored as JSONB arrays), `risk.betti_sequence` (β_0, β_1, … across thresholds), `risk.persistence_landscape` (the functional summary, as arrays), and `risk.topological_risk_score` (one composite score per window). Storing each stage — not just the final score — is what lets the dashboard show *why* the score moved and lets `regime.transition` detection compare consecutive windows' landscapes with a proper distance. The correlation matrices these windows are built from are Market Data's, fetched over its API in **one batched request covering the whole window range** rather than one call per window (§5.4), and **fingerprinted into the window row** (basket, range, source matrix hash) so a persistence diagram stays reproducible and attributable to an exact input even though that input lives in another service.

8.11 Connection management, migrations & schema evolution

Every service's data-access layer uses **SQLAlchemy 2.0 (async) over asyncpg**, with **PgBouncer** in transaction-pooling mode in front of its database. PgBouncer reuses a small pool of live backend connections instead of opening one per request, which matters because an async API can hold many concurrent logical connections; transaction-pooling keeps the physical backend count bounded and predictable. Under microservices this

matters more, not less: eight services each with their own pool can exhaust a Postgres instance's connection limit far faster than one application could, so each service's pool is explicitly sized and the sum is checked against the instance limit rather than left to chance.

Migrations are managed with **Alembic**, and each service owns **its own independent migration history** in its own repository, applied by its own pipeline. There is no global migration order and no coordinated schema release — that independence is the point (§8.2). Two disciplines keep it safe. Migrations must be **backward-compatible with the currently-running version of their own service**, because a rolling deployment briefly runs old and new code against one database; structural changes therefore follow the **expand/contract** pattern (add the new column and backfill, deploy code that writes both and reads the new, then drop the old in a later release) rather than a single breaking `ALTER`. And a migration may never touch another service's database, which the credential separation enforces mechanically. The migration history remains the authoritative schema definition (P7) — each database is rebuildable from empty by running that service's migrations, with no manual DDL.

8.12 Cross-service read models (local projections)

Because no service may read another's database (P8) and no view may be assembled with a cross-service join, any service needing another's data on its **read** path maintains a **local projection**: a table in its own database, shaped for its own queries, populated by consuming the owning service's events. This is the mechanism referenced throughout §5, and it is worth specifying once because it is the most common structure in the system after the outbox.

Consuming service	Local projection	Fed by	Why it exists
Dashboard	Widget read models — portfolio overview, drawdown series, stress summaries, regime & topological-risk views	`portfolio.*`, `fill.*`, `stress.completed`, `regime.transition`, `prediction.ready`	A widget read must touch one database, not five (§5.10)
ML	Feature store values with `knowledge_time`	`bar.ingested`, `news.ingested`, `portfolio.*`, `fill.*`	Look-ahead safety needs the time a fact *became known*, which only the event carries (§8.6)
Risk	Position snapshots and instrument reference data	`portfolio.*`, `instrument.*`	A stress run must not fail because Trading is redeploying
Simulation	Instrument reference data	`instrument.*`, `instrument.retired`	Display and validation without a synchronous hop per run
XAI	Prediction and stress-result summaries	`prediction.ready`, `stress.completed`	Reactive by design; the trigger and its context arrive together
Platform	Cross-service audit index	`audit.recorded`	Answers "what did this user change anywhere" without owning the trail (§8.7)

Four rules govern every projection, and they are what keep this from degenerating into eight copies of the truth. **A projection is never authoritative** — it is a cache of another service's fact, and any write path that would depend on its accuracy must validate against the owner instead (§8.9). **A projection stores only the fields the consumer actually reads**, because a projection that mirrors the source table is a shared schema in disguise and re-couples the two services' models. **Every projection is rebuildable from the event stream**, so a corrupted or newly-added projection is repaired by replaying from the retained log rather than by a manual data fix — which is a direct benefit of Kafka retention and of the outbox guaranteeing the stream is complete. And **every projection's staleness is measured**, exposed as consumer lag per service (§9.5), so "how out of date can this widget be?" has a number rather than an assumption.

9. Cross-cutting concerns

9.1 Authentication. The Platform service is the sole issuer of tokens; every other service is a **verifier**. Tokens are signed **asymmetrically** (RS256/EdDSA) and each service verifies the signature locally against Platform's published public key, cached from a JWKS endpoint. This is the load-bearing choice: a symmetric secret would force every service to hold the signing key, and calling Platform to validate each request would make it a synchronous dependency on every hop in the system and a single point of failure for all traffic. Local verification means Platform can be briefly unavailable without any authenticated request failing.

The cost is that a stateless token cannot be un-issued, which matters because an Admin deactivating a user must take effect promptly (FR-PLT-002). Three mechanisms together bound the exposure: access tokens are **short-lived** (minutes), refresh tokens are stored server-side in `platform_db` and are checked on refresh so a deactivated user cannot obtain a new access token, and Platform publishes `user.deactivated` and `role.changed` events that every service consumes into a small local **denylist** checked during verification. The honest statement of the guarantee is therefore "revocation propagates within the access-token lifetime, and immediately for services that have consumed the event" — not "instantly everywhere", which a distributed system cannot promise without the per-request call this design deliberately avoids.

9.2 Authorization (RBAC). Authentication happens once at the edge; **authorization happens in the service that owns the action**, on every route (P4, security NFR), keyed on the four roles — admin, quant, analyst, viewer. The gateway verifies the token and forwards a verified identity inward; it does **not** decide whether the action is permitted, because a gateway-only check would make every service implicitly trust anything that reached it, and internal traffic (event consumers, service-to-service calls on a user's behalf) never passes through the gateway at all. Each service declares the required role on each route through the shared template's RBAC dependency, so "authorize everywhere" is structural rather than a discipline each developer must remember. The role set is additionally pinned in `platform_db` (§8.9).

Row-scoped access (which portfolios and instruments a user may see) is owned by Platform but enforced by each service in its repository layer via mandatory scope filters. Scope is delivered two ways to avoid a per-request call: coarse scope travels in the token's claims, and services that need finer or frequently-changing grants maintain a local projection of them fed by Platform's events (§8.12). A service that cannot resolve scope **denies** rather than defaulting to open — the failure mode of an authorization check is always closed.

9.3 Secrets & encryption. No secret is ever in source or an image. **Each service has its own credentials** — its own database role, its own object-store prefix credentials, its own Kafka client identity — issued from **HashiCorp Vault**, so a compromise is bounded to one service's data rather than the platform's. Vault's dynamic database credentials are used where practical so a service's database password is short-lived and

never long-lived in an environment variable. Third-party keys (the LLM provider) are held only by the one service that calls out. Development uses a git-ignored `.env` per service, explicitly marked dev-only. All traffic is encrypted in transit: TLS on client↔gateway, and on every service↔service and service↔datastore connection — inside the cluster as well as at the edge, because "internal" is not a trust boundary when eight services share a network.

9.4 Caching. Three layers, each owned by exactly one service. TimescaleDB continuous aggregates (materialized, in the owning service's database) for time-series roll-ups; Redis for hot payloads and expensive query results, in that service's own logical database with explicit TTLs; and HTTP-level caching on Market Data's immutable historical reads (ETags and long-lived cache headers on closed windows), which is what prevents four services' feature computations from repeatedly re-fetching the same bars. A service invalidates only its own cache, on its own writes or on the events it consumes. Cache is an accelerator over a correct source, never a system of record.

9.5 Observability. Observability stops being a convenience under microservices and becomes the only way the system is debuggable at all, because a single user action now spans a gateway, two or three services, a queue, a worker, and several Kafka topics, and no single log file contains the story.

Distributed tracing is therefore mandatory, not optional: every service is instrumented with **OpenTelemetry**, a trace context is created at the gateway and propagated across every HTTP call, every job enqueue, and every Kafka message header, so one trace shows the full causal path with per-hop latency. The correlation id travels in the same context, and structured JSON logs from all eight services ship to one searchable store keyed on it — a request that fails three services deep is found by one query rather than by reading eight logs and guessing. **Metrics** are collected per service and include the distributed-specific signals that would not exist in a monolith: per-dependency call latency and error rate, circuit-breaker state, retry counts, saga completion and compensation counts, **Kafka consumer lag per consumer group** (which is the direct measure of read-model staleness, §8.12), outbox drain backlog and age of the oldest unpublished row, and queue depth per service. A service-level dashboard per service plus a platform overview is the operating surface. Because a trace can end minutes later on a worker, span links join the asynchronous continuation to the originating request rather than leaving two unrelated traces.

9.6 Push delivery. The Dashboard service owns all client connections (§7.3). It consumes `risk.threshold.breached`, `portfolio.*`, `sim.progress`, and the other domain events, evaluates user alert rules, records every trigger including acknowledgement in `dashboard_db`, and pushes notifications to connected clients over WebSocket/SSE. The user never polls (accessibility NFR). Plain-language explanation text is attached where available — Dashboard consumes `explanation.ready` from XAI and includes the narrative if it has arrived, and sends the alert immediately without it if not, so a slow LLM provider delays the prose rather than the warning.

9.7 Configuration. Twelve-factor, per service: all configuration is injected via environment or secret manager, the same image runs in every environment, and nothing about where a service runs is baked into its build. Each service is configured independently — there is no shared configuration file that would couple eight services together.

10. Physical deployment architecture

Per assumption **A2**, production targets Kubernetes and development runs the whole platform on one host under Docker Compose. The two topologies run the same images and the same code; they differ in replica counts, in whether infrastructure is clustered, and in the two co-location concessions A2 records.

10.1 Development / demo — Docker Compose (single host)

One `docker-compose.yml` brings up the gateway, all eight service APIs, and the seven workers those services ship (§5.2) — sixteen application containers in total — alongside one PostgreSQL container holding the eight logical databases with eight separate roles, one Redis with per-service logical databases, a single-broker Kafka with the schema registry, MinIO, and the SPA served statically. Every worker is declared here rather than folded into its API, because a worker that only exists in production is a code path that never gets tested; running them separately on a laptop is what makes the demo exercise the real topology, including the outbox drain, saga compensation, and per-service migrations. Workers are grouped under Compose *profiles* so a developer working on one slice can start a subset without editing the file (§15 records the resource budget this trades against).

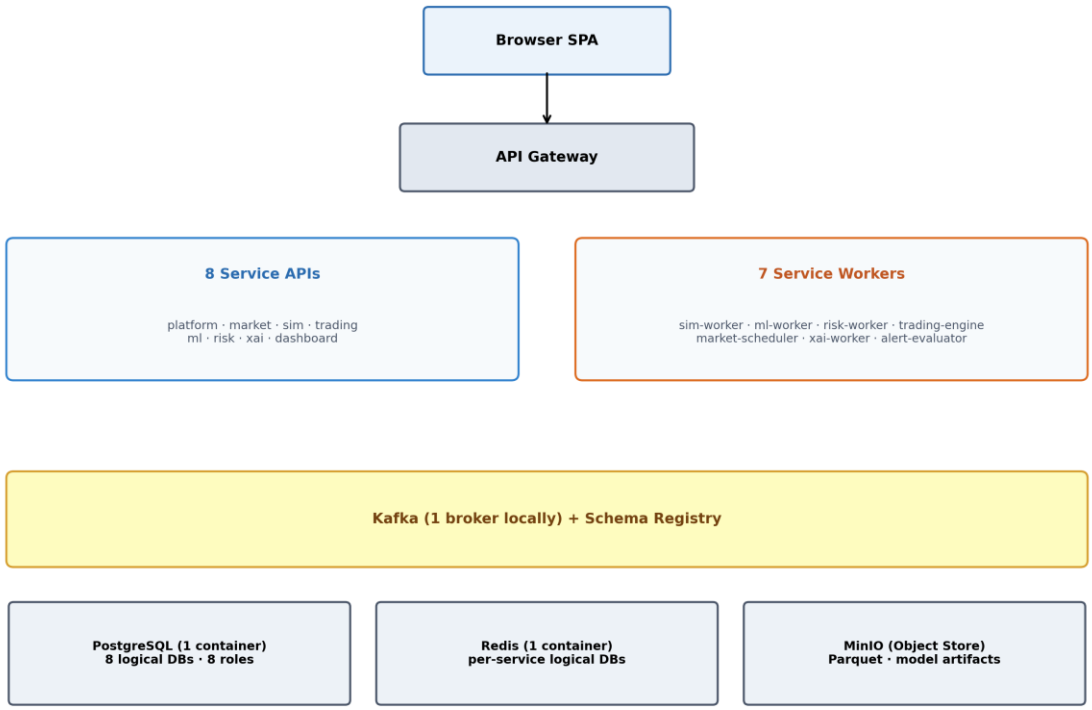


Figure 10.1: Development / Demonstration Single-Host Docker Compose Topology

10.2 Production — Kubernetes

Each service is a **Deployment** with its own image, its own replica count, its own resource requests and limits, liveness and readiness probes, and a **HorizontalPodAutoscaler** where its load varies; each is reached in-cluster by a stable `Service` DNS name (§7.8). Compute workers are separate Deployments from their service's API, and the ML worker Deployment carries a GPU node selector and tolerations so only it is

scheduled onto GPU nodes. The gateway sits behind an **Ingress** with TLS termination and is the only externally routable component. Databases run as a managed Postgres instance per service (or per-service instances in one managed cluster), each with PgBouncer and, where §8.8 justifies it, a read replica. Kafka runs as a ≥3-broker cluster with the schema registry. Vault provides per-service credentials, and **Terraform** provisions everything while each service's Alembic history provisions its own schema — so the environment is rebuildable from code with no engineer's manual knowledge in the loop (P7, recovery NFR).

Two deliberate omissions. There is **no service mesh**: mTLS, retries, and circuit breaking are handled in the shared service template (§5.1, §7.5), which is sufficient at eight services and avoids a substantial operational and debugging burden; a mesh becomes worth reconsidering if the service count grows or if traffic policy needs to change without redeploying, and §15 records it as an open question rather than a settled no. And there is **no separate service registry**, because Kubernetes DNS already is one.

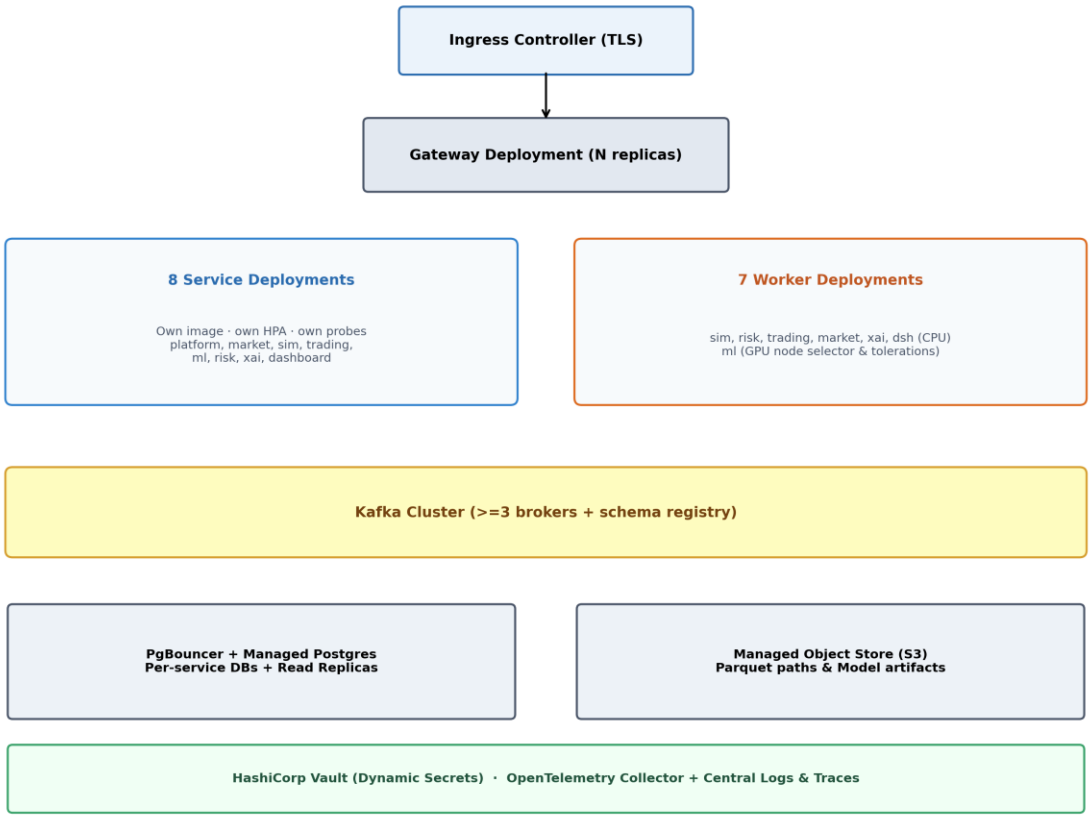


Figure 10.2: Production Kubernetes Deployment Architecture

10.3 Scaling knobs

Every service scales independently, which is the concrete payoff of A1: interactive load → gateway and the affected service's API replicas; simulation throughput → Simulation worker replicas; TDA throughput → Risk worker replicas; ML throughput → ML worker replicas, bounded by the GPU budget; dashboard read load → Dashboard replicas and its read replica; market-data fan-out → Market Data replicas plus HTTP cache hit rate; event throughput → Kafka partitions and brokers, with partition count per topic sized to its consumers' parallelism. Because Trading's correctness depends on per-portfolio ordering (§5.6), it scales by **adding**

partitions and consumer instances that each own whole partitions, never by adding concurrent consumers to one partition — the one place in the system where a scaling knob has a correctness constraint attached, and therefore the one worth stating explicitly.

11. Reliability, recovery & data-durability design

Under A1, reliability has two distinct halves that a monolith conflates: staying correct when a *dependency* fails (partial failure, §11.1), and staying recoverable when *storage* fails (durability, §11.2). Both must be designed, because P9 states that partial failure is the normal case, not the exception.

11.1 Partial failure and graceful degradation

Every outbound call is wrapped by the template's resilience chain (§7.5), so a slow or dead dependency is converted into a fast, named failure rather than a spreading stall — this is what stops one service's outage from becoming an outage of all eight. Each dependency has a declared degradation behaviour, and the rule is that a service degrades to a *narrower but still honest* answer rather than returning a wrong one:

If this is unavailable	The affected service	Degrades to
Market Data (replay/quotes)	Simulation, Trading, Risk	Reject new runs with a clear cause; in-flight runs continue on already-fetched windows
Simulation	Risk, Dashboard	Risk serves metrics for previously completed runs; Dashboard marks the run panel stale, not empty
ML	Trading (signals, *optional*), XAI	Trading falls back to the strategy's rule-based branch or to manual entry — an ML-consuming strategy is configured with that fallback or it may not be enabled (§5.2); XAI returns "explanation unavailable", never a fabricated one
Risk & TDA	Dashboard, Alerting	Last computed metrics served with an explicit `as_of` age; breach alerts suppressed rather than guessed
XAI	Dashboard	Metrics render without narration (numbers are never withheld because prose failed)
Dashboard	Browser SPA	Widgets show last-known values with staleness badges; the SPA falls back to polling if the socket drops
Platform / Identity	All services	Requests keep working for the life of already-issued tokens (§9.1); only login and role changes stall
Kafka	All publishers	Writes still commit locally; events accumulate in the outbox and drain when the broker returns (§7.7)

The last two rows are the ones that matter most, because they are where a naive decomposition would have created a single point of failure for the whole platform and this design does not.

11.2 Durability, per service

Each service database is backed up independently: continuous WAL archiving to object storage plus periodic base backups via **pgBackRest**, giving an RPO well under five minutes and recovery to any moment rather than to the last nightly snapshot. Where §8.8 justifies a read replica, primary failure is handled by promoting it. Backups are proven by **scheduled restore drills** into a throwaway environment — a backup is not valid until a restore has succeeded, so "we have backups" is never mistaken for "we can recover." **Event durability** comes from Kafka's replication factor ≥ 3 with `min.insync.replicas = 2` and `acks=all`, so losing one broker loses no events; topic retention is set long enough that a consumer can be rewound to rebuild a read model (§8.12) rather than only to catch up.

Because there are eight databases, recovery has a property a single database does not: **restores are per service and therefore not mutually consistent.** Restoring `risk_db` to 14:00 while `simulation_db` is current leaves Risk missing results for runs Simulation knows finished. The design accepts this instead of pretending to coordinate eight PITR timelines, and handles it the same way it handles every other consistency gap: after a restore, the service **replays the retained event stream from its restored offset** and reconciles (§8.9). This is why read models must be rebuildable from the stream (§8.12) and why consumers must be idempotent (§11.3) — those two rules are what make single-service recovery possible at all.

11.3 Correctness under retry

At-least-once delivery means duplicates are certain, not hypothetical, so every consumer is idempotent: it records handled events in `processed_event` and treats a repeat as a no-op (§7.7). Sagas compensate rather than roll back, and every compensating action is itself idempotent and safe to re-run, because a compensation can be interrupted too. A failed step that cannot be compensated automatically ends in an explicitly recorded `*.failed` state with the reason attached, so the system never leaves a saga silently half-finished.

11.4 Full rebuild

Terraform plus the eight independent Alembic histories reconstruct the entire platform from nothing (P7). Because each service owns its own migration history, a rebuild is also a per-service operation — a single service can be recreated and re-seeded from the event stream without touching the other seven.

12. Non-functional design — how each NFR is met (traceability)

Every architectural mechanism above exists to serve a requirement; this matrix is the audit trail from NFR to mechanism, and it is where "is this design optimal?" is answered concretely.

NFR theme	Requirement (paraphrased)	Mechanism in this design
Performance	Dashboard widgets < 2 s	Dashboard read models answered from its own DB — no fan-out to five services per widget — plus continuous aggregates, Redis cache, replica reads (§5.10, §8.4, §8.12, §9.4)
Performance	CRUD < 800 ms	Async FastAPI + per-service PgBouncer + UUIDv7 index-

NFR theme	Requirement (paraphrased)	Mechanism in this design
		friendly keys; no heavy work on any API tier (§6, §8.11)
Performance	Audit overhead < 20%	Lean single-insert trigger, local to each service DB; audit table compressed hypertable; per-table opt-in (§8.7)
Performance	One slow dependency does not slow the caller	Per-call timeout + circuit breaker + bulkhead — failure is fast and named (§7.5)
Scalability	Simulation scales without hurting the API	Simulation workers are a separate deployment from every API (§6, §10.2)
Scalability	Each capability scales independently	One deployment per service, own replica count and HPA (§10.2, §10.3)
Scalability	Reads scale independently of writes	Two-level read/write separation: read models across services, replica routing within one (§8.8, §8.12)
Scalability	New model/scenario without schema change	Registry rows + JSONB config, not enums/tables (P5, §8.6)
Reliability	Dependency outage degrades, not cascades	Declared degradation per dependency; resilience chain on every call (§7.5, §11.1)
Reliability	Replica fallback	Streaming replication + promotion, per service database (§8.8, §11.2)
Reliability	PITR RPO < 5 min	WAL archiving + pgBackRest base backups, per service database (§11.2)
Reliability	Backups verified by drills	Scheduled restore into non-prod (§11.2)
Reliability	Kafka survives one broker loss	RF ≥ 3, min.insync=2, acks=all (§7.4, §11.2)
Reliability	No event lost when the broker is down	Transactional outbox drains after recovery; local commit never blocked (§7.7, §11.1)
Reliability	Duplicate delivery cannot corrupt state	Idempotent consumers + `processed_event` dedup; idempotent compensations (§7.7, §11.3)
Integrity	Monetary precision	`NUMERIC` with explicit scale in every service database (§8.9)
Integrity	Role constrained at DB	`ENUM` role type in `platform_db` (§8.9)
Integrity	Reject inverted replay window	`CHECK (end_time > start_time)` (§8.9)
Integrity	Cross-service references stay sound without FKs	Validate on write, react to invalidation events, reconcile periodically (§8.9)

NFR theme	Requirement (paraphrased)	Mechanism in this design
Integrity	Multi-service operations end in a consistent state	Choreographed sagas with compensating actions; no partial success exposed (§7.6)
Security	Authz on every route	Authentication at the gateway, authorization in the owning service, fail-closed (§9.1, §9.2)
Security	Encrypt all traffic	TLS at the edge and on every internal hop (§9.3)
Security	Secrets in a manager	Vault, with credentials issued per service (§9.3)
Security	Compromise of one service is contained	Per-service DB roles and Vault credentials; no shared superuser (§8.2, §9.3)
Security	Passwords hashed	Argon2/bcrypt hashes in `platform_db`; never plaintext (§8.2)
Security	Audit survives row deletion	`before` image captured on delete (§8.7)
Recovery	Rebuild from IaC + migrations	Terraform + eight independent Alembic histories (§10.2, §11.4, P7)
Recovery	A single service can be restored alone	Per-service backups + event replay from retained offset (§11.2)
Audit	Every write recorded before/after	Generic audit trigger in each service database (§8.7)
Audit	Reconstruct any record's history	Query the owning service's `change_log` by `row_pk` ordered by time; cross-service index via `audit.recorded` (§8.7)
Observability	A request can be followed end-to-end	OpenTelemetry trace context propagated across HTTP, queue, and Kafka hops; span links for async continuations (§9.5)
Observability	Staleness of derived views is measurable	Kafka consumer lag per group + outbox drain backlog as first-class metrics (§8.12, §9.5)
Maintainability	Deploy one capability without redeploying others	One image and pipeline per service; additive-only contract changes with URL versioning (§7.1, §10.2)
Maintainability	Contract breakage caught before deploy	Consumer-driven contract tests run in the provider's pipeline; schema registry (§7.1, §7.4)
Accessibility	Plain-language beside metrics	XAI narration attached to widgets/alerts; numbers still render if XAI is down (§5.9, §9.6, §11.1)
Accessibility	Push, not manual refresh	Dashboard-owned WebSocket/SSE delivery with polling fallback (§9.6, §11.1)

NFR theme	Requirement (paraphrased)	Mechanism in this design
ML correctness	No look-ahead bias	Bi-temporal feature store materialized from events with `knowledge_time` stamped at arrival; as-of reads only (§5.8, §8.6)

13. Technology stack — and why each choice is the optimal one

Layer	Choice	Why optimal here	Considered instead
Frontend	React + TypeScript, ECharts/Plotly	Rich charting ecosystem; strong typing for a data-dense UI	Vue/Svelte (smaller ecosystem for finance charts)
Edge	API gateway (Kong/Traefik, or an in-house FastAPI edge)	One TLS/authN/rate-limit/aggregation point so eight services aren't eight public surfaces (§5.2, §9.1)	Expose each service publicly (duplicated auth, CORS sprawl); heavyweight mesh ingress (unneeded at this size)
Services	Python 3.12 + FastAPI (async), one image per service	Same language as the quant/ML code (no rewrite across a boundary); async fits I/O concurrency; Pydantic *is* the validation layer; auto OpenAPI gives the per-service contract for free	Node/Go per service (would split the codebase from the ML/quant work for no gain the team needs)
Service scaffolding	Shared internal template/library (§5.1)	The single most important answer to "eight services, three people" — resilience, tracing, outbox, and migrations are written once	Each service hand-rolled (eight divergent implementations)
Sync inter-service	REST/JSON over HTTP; gRPC where a hot path justifies it	OpenAPI-documented, trivially debuggable; gRPC reserved for measured hot paths rather than adopted everywhere	gRPC for all calls (tooling cost, weaker browser story); GraphQL federation (extra runtime to operate)
Async runtime	asyncio + Arq (or Celery) workers, one queue per compute service	Job semantics (retry, lease, dead-letter) that a log doesn't provide; queues stay service-local (§7.2)	Threads (GIL-bound for CPU); bare Kafka as a job queue (no lease/retry semantics)
Event backbone	Apache Kafka + schema registry, in every environment	Durable, partitioned, replayable, HA — meets the no-loss NFR; per-portfolio ordering by partition key is exactly what Trading needs; identical in dev and prod so ordering and lag bugs surface locally	RabbitMQ (weaker log/replay for tick streams); Redis Streams in dev only (withdrawn — a

Layer	Choice	Why optimal here	Considered instead
			dev/prod substitution hides the bugs that matter most)
Cross-service consistency	Transactional outbox + choreographed sagas	Restores atomicity between a local commit and a published event without a distributed transaction; sagas keep multi-service flows recoverable (§7.6, §7.7)	Two-phase commit (poor fit, poor support); dual writes (silent divergence); an orchestrator service (extra coupling and a new SPOF)
Data ownership	One PostgreSQL 16 + TimescaleDB database per service	Transactions, constraints, JSONB *and* time-series in one engine, while P8 keeps each service's storage private	Shared database (the classic distributed-monolith mistake); a different engine per service (unjustified operational load)
Pooling	PgBouncer per service (transaction mode)	Bounds physical connections when many replicas each hold a pool	Per-process pools only (connection exhaustion at replica count)
Cache/queue	Redis, one logical DB per service	One system covers cache and job-queue backend; logical separation preserves ownership	A shared keyspace (accidental cross-service coupling)
Cold store	S3-compatible object store + Parquet; MinIO locally	Cheap columnar cold data + model artifacts; same API local and cloud	Keep everything in PG (expensive, slow to back up)
Persistent homology	Ripser / GUDHI (+ persim for landscapes)	Fast, standard, well-validated PH libraries	Hand-rolled PH (error-prone, slow)
ML	scikit-learn + XGBoost + PyTorch/HF Transformers	Covers tabular, boosted trees, LSTM, and financial-NLP in one ecosystem	—
Model tracking	Registry tables in `ml_db` + artifact store (MLflow-compatible)	Domain metadata queryable where it is owned; binaries in object store; predictions attributable	Files on disk (not reproducible/attributable)
Migrations	Alembic, an independent history per service	Schema-as-history enables rebuild-from-code; expand/contract keeps deploys independent (§8.11)	One shared migration history (couples eight deploy pipelines); manual DDL (violates P7)
Contract testing	OpenAPI + consumer-driven contract tests in provider pipelines	Catches a breaking change before it reaches a consumer — the failure mode decomposition introduces (§7.1)	Integration tests only (slow, and they catch it too late)

Layer	Choice	Why optimal here	Considered instead
Observability	OpenTelemetry SDK + collector; centralized logs and metrics	A request crossing four services is untraceable without it; mandatory, not optional (§9.5)	Per-service logs only (no cross-service causality)
Orchestration	Kubernetes (prod) / Docker Compose (dev)	Per-service replicas, probes, HPAs, GPU node selection; Compose keeps the same images runnable on a laptop	Nomad/Swarm (smaller ecosystem); VMs per service (manual scaling)
Secrets	HashiCorp Vault	Named in NFRs; issues distinct dynamic DB credentials per service (§9.3)	Env vars in prod (insecure; and one leak would expose all eight databases)
IaC	Terraform	Whole environment — eight services, eight databases, cluster, broker — reproducible from code	Manual provisioning (violates P7, and unmanageable at this component count)

14. Key architectural decisions (ADR summary)

ID	Decision	Rationale	Rejected alternative
AD-1	**Microservices — one service per bounded context** (8 services + gateway)	Team decision (A1): capabilities deploy, scale, and fail independently, and the boundaries are already clear from the domain	Modular monolith (cheaper to operate, but gives up independent deploy/scale/failure); finer or coarser splits (§3)
AD-2	Background work deployed separately from each service's API (**7 workers across 8 services**)	Long jobs must never block requests, and CPU/GPU capacity must scale on its own axis; job workers scale on backlog, continuous workers on consumer lag (§6)	Run jobs inline in the API; one shared worker fleet for the platform
AD-3	**Database per service; no shared tables, no cross-service FKs or joins**	Ownership is what makes independent deployment real; one convenient shared query turns eight services into a distributed monolith (P8)	Shared database with a schema per service
AD-4	Task queue ≠ event backbone, and each queue is service-local	Job semantics and stream semantics are different problems; a shared queue would recouple the services	One Kafka for everything; one shared queue
AD-5	**Transactional outbox for every publish**	Makes "commit locally and publish" atomic without a distributed transaction; converts an unsolvable problem into	Publish inside the transaction (lost events); publish after commit (silent divergence)

ID	Decision	Rationale	Rejected alternative
		"tolerate duplicates"	
AD-6	**Choreographed sagas with compensating actions**	Multi-service flows stay recoverable and no service depends on a central coordinator	Two-phase commit; a central orchestrator (new coupling and SPOF)
AD-7	**Asymmetric JWT (RS256/EdDSA) verified locally against JWKS**	Platform/Identity never becomes a synchronous dependency of every request; revocation handled by an event-driven denylist	Opaque tokens introspected per request (SPOF + latency on every call)
AD-8	**Local read models fed by events, never authoritative**	Dashboard answers a widget from one database instead of fanning out to five services; staleness is measured, not hidden	Synchronous cross-service reads per widget; caching another service's DB
AD-9	Event sourcing in Trading, private to `trading_db`	Exact replay is a stated requirement, not derivable from current-state storage; no other service reads the log	Store current state only; expose the log as a shared read surface
AD-10	Bi-temporal feature store, materialized from events	Records when a fact *became known*, making look-ahead bias structurally impossible — decomposition improves this rather than costing it	Trust modellers to avoid leakage; join a mutable table
AD-11	Registry + JSONB for open catalogs	Add model/scenario types without a migration	Enum/table per type
AD-12	TimescaleDB over a separate TSDB	Time-series and relational integrity in one engine, per service	Postgres-only or a dedicated TSDB
AD-13	Invariants in each service's database	A second code path cannot bypass a DB constraint; cross-service invariants degrade to validate/react/reconcile (§8.9)	Application-only checks
AD-14	Resilience chain (timeout, bounded retry, circuit breaker, bulkhead) in the shared template	Under P9 the network is a normal failure source; written once so all eight services inherit it	Per-service ad-hoc handling; a service mesh (deferred, §15)
AD-15	Kafka in **every** environment, no dev substitute	Ordering, lag, and rebalance bugs must surface locally; a dev-only backbone hides exactly the class of bug decomposition introduces	Redis Streams locally behind an interface (the 0.1 position — withdrawn)
AD-16	Kubernetes in prod, Docker Compose in dev, same images	Per-service replicas/probes/HPAs/GPU scheduling in prod; whole platform still runs on a laptop for the demo	Full cloud now (cost/time); Compose in prod (no per-service scaling)

ID	Decision	Rationale	Rejected alternative
AD-17	Independent Alembic history per service + expand/contract	Two services must never have to deploy together for a schema change	One shared migration history
AD-18	**Authorization decided in the service that owns the action**, authentication once at the edge	Internal traffic (event consumers, service-to-service calls) never passes the gateway, so a gateway-only check would make every service implicitly trust whatever reached it (§9.2)	All authorization at the API layer
AD-19	**Rolling correlation matrices owned and served by Market Data**, fetched by Risk in one batched call and fingerprinted	Correlation is derived market data; ML needs the same series; a closed-window matrix is immutable so compute-once beats per-consumer recomputation (P6, §5.4)	Risk recomputes correlations from bars it fetches (two implementations of one definition, guaranteed to drift)
AD-20	**Cleaning and normalization gate publication** — nothing enters published tables or the event stream until it passes, and each run emits a reconciliation report	A published bad bar cannot be retracted cheaply: every downstream read model has already projected it and every derived feature is silently wrong (§5.4, FR-MKT-011)	Clean on read, or clean after publication

15. Open questions & assumptions to confirm

These are the genuine ambiguities the design brackets; each needs a team/advisor decision and none blocks starting.

Local resource budget for eight services. Compose must run eight service APIs plus seven worker deployments (§5.2), Postgres, Kafka, the schema registry, Redis, and MinIO on one development machine. A2 already concedes a shared Postgres instance and a single broker for this reason, but the memory ceiling on the team's actual hardware needs measuring — and if it binds, the fallback is a documented Compose profile that starts a subset of services and workers for a given task rather than silently changing the architecture.

Orchestration platform in production. §10.2 targets Kubernetes generically; whether that is a managed cluster, a lightweight distribution (k3s), or a single-node cluster for the demo affects nothing architecturally but changes the cost and setup effort materially.

Service mesh. Deferred in §10.2 because the shared template already provides mTLS, retries, and circuit breaking at eight services. Worth revisiting if the service count grows or traffic policy needs to change without redeploying.

Saga coordination style if flows deepen. Choreography (AD-6) is right for the current two- and three-step flows. If a flow reaches five or more steps with conditional branches, the distributed logic becomes hard to follow and an orchestrator for that specific flow may be the better trade — a decision to make on evidence, not in advance.

Contract-test tooling. Consumer-driven contract testing is committed to (AD-1's cost, §7.1); whether that is Pact, schemathesis against each OpenAPI document, or generated client fixtures is unsettled and depends on what the team can maintain.

Corporate actions. Splits, mergers, and dividends distort raw price series and must be handled (adjusted vs unadjusted series). Market Data reserves a `corporate_action` table and an adjustment step in ingestion, but the policy — store both series, or adjust on read — is unresolved, and because the adjusted series is published as events every downstream read model inherits the choice.

GPU budget & ownership. The ML worker deployment assumes a bounded, shared device budget; how many concurrent trainings are allowed, and on whose hardware in the demo, needs a number.

Real-time vs pseudo-real-time data source. "Pseudo real-time replay" implies historical data played back, not a live paid market feed. Confirm no live feed is in scope (it changes ingestion and cost materially).

LLM provider for XAI narration. External API (hosted) vs a local open model changes the secret-management, latency, and cost story in §9.

Scale targets for sizing. Concrete numbers — instruments tracked, max $N \times T$ per simulation, retention horizon, and Kafka retention long enough for the read-model rebuilds §11.2 depends on — would let §8.4 partitioning and §10.3 scaling be sized rather than described.

16. Summary

QuantSim is architected as **eight independently deployable microservices behind an API gateway, integrated by a Kafka event backbone**. Platform/Identity, Market Data, Simulation, Trading, ML, Risk & TDA, Explainability, and Dashboard & Alerting each own one bounded context, one database, and one deployment pipeline — one service per module named in the approved proposal, plus Platform/Identity, which exists precisely so that identity and audit concerns do not distort any of the seven. Seven of the eight run their background work as a separate worker deployment (§5.2), so CPU and GPU capacity scale on their own axis and interactive latency is never held hostage to a long job. This is a deliberate team decision (A1), taken for independent deployability, independent scaling and failure isolation, with its costs — distributed data, eventual consistency, the network as a normal failure source, and an eight-service operational surface — accepted knowingly rather than argued away.

The consequences of that decision are designed for rather than discovered later. Data ownership is absolute (P8): no shared tables, no cross-service joins or foreign keys, so integrity that a single database would have enforced with a constraint is replaced by validate-on-write, react-to-invalidation, and periodic reconciliation — weaker, and documented as weaker. Multi-service operations run as choreographed sagas with compensating actions, and every publish goes through a transactional outbox so a local commit and its event can never diverge; consumers are idempotent because at-least-once delivery makes duplicates certain. Cross-service reads are served from local read models fed by events, never authoritative and always measurably stale, which is what lets a dashboard widget answer from one database instead of fanning out to five services. Partial failure is the normal case (P9): every outbound call carries a timeout, bounded retry, circuit breaker, and bulkhead, and every dependency has a declared degradation behaviour, so an outage narrows what the platform can honestly say rather than cascading.

Within each service the storage strategy is unchanged in ambition and sharper in ownership: PostgreSQL 16 + TimescaleDB for hot operational and time-series data, Redis for cache and job queueing, object-stored

Parquet for cold data, and the database — not the application — enforcing the domain's correctness invariants. Trading's append-only event log gives exact replay and is private to `trading_db`; ML's bi-temporal feature store is materialized from events, which stamps the moment a fact became known and makes look-ahead bias structurally impossible rather than merely discouraged; money precision, role constraints, and a delete-surviving audit trail live in each service's schema. Authentication happens once at the gateway with asymmetric JWTs verified locally against JWKS, so Identity is never a synchronous dependency of every request, while authorization stays in the service that owns the data.

The eight-service operational cost is answered concretely by a shared service template that implements the resilience chain, outbox, tracing, and migration scaffolding once, and by mandatory OpenTelemetry tracing without which a request crossing four services would be untraceable. Production targets Kubernetes with per-service replicas, probes, autoscalers, and GPU node selection; development runs the same images on one Docker Compose host, with only two honest concessions (a shared Postgres instance holding eight privilege-separated databases, and a single Kafka broker). Every non-functional requirement traces to a specific mechanism (§12), every significant choice records the alternative it beat (§13, §14), and the questions the design deliberately leaves open are listed rather than hidden (§15) — so the reasoning can be challenged on its merits, which is the point of writing it down.